



Université Sultan Moulay Slimane

Faculté Polydisciplinaire de Béni Mellal

Département des technologies nouvelles



PROJET DE FIN D'ÉTUDES

Présenté pour l'obtention du diplôme de

Master en Data Science et Sécurité des Systèmes d'Information

Conception et développement d'une plateforme de FabLab virtuel intelligent intégrant la simulation 3D et le Machine Learning

Présenté par :

M. ESSAID ABDELKHALEK

Sous la direction de :

Pr. Soufiane BELHOUIDEG

Organisme d'accueil : École Nationale des Sciences Appliquées de Béni Mellal

Cadre : Projet de Fin d'Études – Virtual FabLab

Soutenu le : À compléter

Devant le Jury :

Pr. Nom Prénom — FP Béni Mellal

Pr. Nom Prénom — FP Béni Mellal

Pr. Nom Prénom — FP Béni Mellal

DÉDICACE

Je dédie ce travail à mes parents, pour leur soutien constant, leurs sacrifices et leurs encouragements tout au long de mon parcours.

À ma famille, pour sa présence, sa compréhension et l'appui qu'elle m'a toujours apporté.

À mes enseignants, pour les connaissances transmises, l'encadrement assuré et la confiance accordée durant ma formation.

À toutes les personnes qui m'ont soutenu, conseillé ou encouragé au cours de mon parcours académique.

Que cette réalisation soit le témoignage de ma reconnaissance envers tous ceux qui ont contribué, de près ou de loin, à mon évolution personnelle et universitaire.

REMERCIEMENTS

Je tiens tout d’abord à exprimer ma profonde reconnaissance à mon encadrant pédagogique pour son orientation, ses conseils avisés, sa disponibilité et son soutien continu tout au long de la réalisation de ce projet de fin d’études. Son accompagnement m’a permis de mieux structurer mon travail, d’approfondir ma réflexion et d’avancer avec rigueur dans les différentes étapes de ce projet.

J’adresse également mes sincères remerciements à l’ensemble du corps professoral et administratif de la Faculté Polydisciplinaire de Béni Mellal (FPBM), pour la qualité de la formation dispensée, l’encadrement assuré et les conditions favorables offertes durant mon parcours académique. Je remercie de même le personnel enseignant et administratif de l’École Nationale des Sciences Appliquées de Béni Mellal (ENSA BM), pour l’accueil, l’accompagnement et les moyens mis à disposition dans le cadre de mon stage et de la réalisation de ce travail.

Mes remerciements s’adressent aussi à l’Université Sultan Moulay Slimane, institution au sein de laquelle s’inscrit cette formation, pour son rôle dans le développement des compétences scientifiques et professionnelles des étudiants. Je tiens également à remercier toutes les personnes ayant contribué, de près ou de loin, au bon déroulement de mon stage et à la réussite de ce projet, par leurs échanges, leurs remarques ou leur soutien.

Enfin, j’exprime toute ma gratitude à ma famille pour son soutien constant, sa patience et ses encouragements tout au long de mon parcours. Que toutes les personnes qui ont contribué directement ou indirectement à l’aboutissement de ce travail trouvent ici l’expression de mes sincères remerciements.

RÉSUMÉ

Ce projet de fin d'études porte sur la conception et le développement d'une plateforme virtuelle de type FabLab intégrant la simulation 3D, l'Internet des Objets et un module Data Science et Intelligence Artificielle. La solution proposée permet de visualiser et de superviser des machines telles qu'une imprimante 3D et une machine CNC dans un environnement interactif.

La plateforme repose sur une architecture moderne combinant une interface web, un backend applicatif, une simulation 3D, une télémétrie simulée via MQTT et un module d'analyse de données. Ce module réalise une détection d'anomalies avec Isolation Forest, une estimation du risque de maintenance avec Random Forest, une classification supervisée des intentions administrateur par TF-IDF et Logistic Regression, ainsi qu'un Copilote IA de maintenance réservé à l'administrateur. Les données de télémétrie utilisées dans cette version sont simulées ; les résultats obtenus doivent donc être interprétés comme une validation de prototype et non comme une validation industrielle.

Mots-clés : FabLab, Industrie 4.0, Simulation 3D, IoT, MQTT, Data Science, Isolation Forest, Random Forest, NLP, Copilote IA, CNC, Imprimante 3D.

ABSTRACT

This final year project focuses on the design and development of a virtual FabLab platform integrating 3D simulation, Internet of Things technologies, Data Science and Artificial Intelligence. The proposed solution enables the visualization and supervision of machines such as a 3D printer and a CNC machine within an interactive environment.

The platform is based on a modern architecture combining a web interface, an application backend, a 3D simulation module, MQTT-simulated telemetry and a data analysis component. This component performs anomaly detection using Isolation Forest, maintenance-risk evaluation using Random Forest, supervised administrator-intent classification using TF-IDF and Logistic Regression, and an admin-only AI Maintenance Copilot. In this version, telemetry data is simulated to validate the architecture, MQTT flows, monitoring and analytics module; the results therefore do not constitute industrial validation.

Keywords: FabLab, Industry 4.0, 3D Simulation, IoT, MQTT, Data Science, Isolation Forest, Random Forest, NLP, AI Copilot, CNC, 3D Printer.

Table des matières

Dédicace	2
Remerciements	4
Résumé	6
Abstract	8
Liste des figures	13
Liste des tableaux	15
Liste des abréviations	17
Introduction générale	19
1 Présentation de l'organisme	20
1.1 Introduction	21
1.2 Présentation de l'ENSA Béni Mellal	21
1.3 Missions et formations	21
1.3.1 Missions de l'établissement	21
1.3.2 Formations proposées	22
1.4 Organigramme de l'établissement	22
1.5 Contexte du stage	23
1.6 Problématique	24
1.7 Objectifs du projet	24
1.8 Présentation générale du Virtual FabLab	25
1.9 Conclusion	26
2 Analyse des besoins et conception	27
2.1 Introduction	28

2.2	Étude des besoins	28
2.3	Besoins fonctionnels	28
2.3.1	Besoins de l'étudiant	28
2.3.2	Besoins de l'administrateur	29
2.4	Besoins non fonctionnels	29
2.5	Acteurs du système	30
2.6	Cas d'utilisation	30
2.7	Architecture générale	31
2.8	Description des modules	32
2.9	Diagramme de classes	33
2.10	Conception du système	34
2.10.1	Conception de l'authentification	34
2.10.2	Conception de la gestion des machines	35
2.10.3	Conception de la réservation	35
2.10.4	Conception de la télémétrie	35
2.10.5	Conception de l'analyse intelligente	35
2.11	Technologies utilisées	36
2.11.1	Next.js	36
2.11.2	FastAPI	36
2.11.3	Three.js	36
2.11.4	Blender	36
2.11.5	MQTT	36
2.11.6	Python	37
2.11.7	Tailwind CSS	37
2.11.8	Scikit-learn	37
2.11.9	Visual Studio Code	37
2.11.10	Technologies et équipements IoT	37
2.12	Conclusion	39
3	Réalisation et implémentation	40
3.1	Introduction	41
3.2	Architecture frontend/backend	41
3.3	Organisation des dossiers	42

3.4	Implémentation du backend	42
3.4.1	API FastAPI	42
3.4.2	Base de données	43
3.5	Système d'authentification	43
3.6	Gestion des utilisateurs et de l'administration	44
3.7	Gestion des machines	44
3.8	Système de réservation	45
3.9	FabLab virtuel en 3D	46
3.10	Simulation CNC	46
3.11	Simulation de l'imprimante 3D	47
3.12	Interprétation des fichiers G-code et NC	48
3.13	Télémetrie simulée avec MQTT	48
3.14	Tableau de bord de supervision	49
3.15	Cartes de prédiction et historique de télémetrie	50
3.16	Intégration technique du Copilote IA	51
3.17	Communication entre frontend et backend	51
3.18	Interfaces réalisées	52
3.19	Conclusion	52
4	Module Data Science et Intelligence Artificielle	53
4.1	Introduction	54
4.2	Architecture du module Data Science et IA	54
4.3	Collecte et nature des données	55
4.4	Prétraitement des données	55
4.5	Ingénierie des caractéristiques	56
4.6	Détection des anomalies par Isolation Forest	57
4.7	Évaluation du risque de maintenance par Random Forest	57
4.8	Classification NLP des intentions administrateur	58
4.9	Copilote IA de maintenance	59
4.10	Intégration dans la plateforme	60
4.11	Évaluation et validation	61
4.12	Limites	62

4.13 Conclusion du chapitre	63
5 Validation et évaluation du système	64
5.1 Introduction	65
5.2 Stratégie de validation	65
5.3 Validation fonctionnelle de la plateforme	65
5.3.1 Validation du frontend	65
5.3.2 Validation du backend	66
5.3.3 Validation de la communication MQTT et de la télémétrie	67
5.3.4 Validation de la simulation 3D	67
5.4 Validation du module Data Science et IA	68
5.4.1 Validation de l’Isolation Forest	68
5.4.2 Validation du Random Forest	69
5.4.3 Validation de la classification NLP	69
5.4.4 Validation du Copilote IA de maintenance	70
5.5 Discussion des résultats	71
5.6 Résultats globaux	72
5.7 Conclusion	73
Conclusion générale	75
Bibliographie et webographie	76

Table des figures

1.1	Organigramme de l'ENSA Béni Mellal	23
1.2	Vue générale prévue de la plateforme Virtual FabLab	26
2.1	Diagramme de cas d'utilisation de la plateforme Virtual FabLab	31
2.2	Architecture générale de la plateforme Virtual FabLab	32
2.3	Diagramme de classes de la plateforme Virtual FabLab	34
3.1	Architecture d'implémentation frontend/backend	41
3.2	Schéma logique de la base de données	43
3.3	Interface de gestion des utilisateurs	44
3.4	Interface de gestion des machines	45
3.5	Interface de réservation des machines	45
3.6	Vue 3D du FabLab virtuel	46
3.7	Simulation d'une trajectoire CNC	47
3.8	Simulation d'un fichier G-code pour imprimante 3D	47
3.9	Processus d'interprétation et de validation des fichiers G-code/NC	48
3.10	Flux MQTT implémenté dans la plateforme	49
3.11	Tableau de bord administrateur et supervision intelligente	50
3.12	Cartes de prédiction et score de santé des machines	50
3.13	Historique des mesures d'une machine	51
3.14	Communication entre le frontend et le backend	52
4.1	Pipeline Data Science et IA de la plateforme	54
4.2	Workflow NLP du Copilote IA de maintenance	59
4.3	Interface flottante du Copilote IA de maintenance	61
5.1	Tableau de bord de supervision et de télémétrie	66

5.2	Communication entre l'interface Next.js et le backend FastAPI	67
5.3	Interface de simulation 3D du FabLab virtuel	68
5.4	Interface du Copilote IA de maintenance réservé à l'administrateur	70

Liste des tableaux

3.1	Organisation générale du projet	42
4.1	Nature des données utilisées par le module Data Science	55
4.2	Entrées utilisées par les modèles Machine Learning	56
4.3	Capacités supportées par le Copilote IA de maintenance	60
4.4	Métriques disponibles pour la classification NLP des intentions	61
4.5	Synthèse de l'évaluation du module Data Science et IA	62
5.1	Résultats d'évaluation du classifieur d'intentions	69
5.2	Synthèse de la validation du système	71

LISTE DES ABRÉVIATIONS

PFE Projet de Fin d'Études

D3SI Data Science et Sécurité des Systèmes d'Information

ENSA École Nationale des Sciences Appliquées

FPBM Faculté Polydisciplinaire de Béni Mellal

IoT Internet des Objets

MQTT Message Queuing Telemetry Transport

CNC Commande Numérique par Calculateur

IA Intelligence Artificielle

API Application Programming Interface

JWT JSON Web Token

JSON JavaScript Object Notation

REST Representational State Transfer

NLP Natural Language Processing

TF-IDF Term Frequency–Inverse Document Frequency

LLM Large Language Model

UI User Interface

UX User Experience

INTRODUCTION GÉNÉRALE

Les avancées technologiques liées à l'Industrie 4.0 transforment progressivement les environnements de fabrication, d'apprentissage et de prototypage. Les FabLabs représentent aujourd'hui des espaces essentiels pour l'innovation, car ils permettent aux étudiants, chercheurs et porteurs de projets d'accéder à des machines de fabrication numérique telles que les imprimantes 3D et les machines CNC.

Cependant, l'utilisation classique d'un FabLab reste généralement limitée par la présence physique, la disponibilité des machines et l'absence d'outils numériques avancés pour la supervision et l'analyse des données. Dans ce contexte, la mise en place d'une plateforme virtuelle devient une solution pertinente pour améliorer l'accessibilité, la visualisation, la gestion et l'exploitation intelligente des équipements.

Le présent projet vise à concevoir et développer une plateforme virtuelle FabLab intégrant une interface web moderne, une simulation 3D interactive, une communication IoT simulée via MQTT et un module Data Science et Intelligence Artificielle destiné à l'analyse des données machines et à l'aide à la maintenance. Ce module combine la détection d'anomalies, l'estimation du risque, la classification supervisée d'intentions et un copilote d'aide à la décision réservé à l'administrateur. Les données utilisées dans cette version sont simulées afin de valider l'architecture, les flux MQTT, le monitoring et le module d'analyse. La plateforme reste conçue pour être connectée ultérieurement à des capteurs réels.

Ce rapport est organisé comme suit. Le premier chapitre présente l'organisme d'accueil ainsi que le cadre général du projet, la problématique et les objectifs visés. Le deuxième chapitre est consacré à l'analyse des besoins, à la conception du système et aux technologies utilisées. Le troisième chapitre détaille la réalisation et l'implémentation effective de la plateforme. Le quatrième chapitre présente le module Data Science et Intelligence Artificielle. Enfin, le cinquième chapitre présente la validation fonctionnelle, les résultats obtenus, les limites actuelles et les perspectives d'évolution de la solution.

CHAPITRE 1

PRÉSENTATION DE L'ORGANISME D'ACCUEIL ET CADRE GÉNÉRAL DU PROJET

1.1 Introduction

Le Projet de Fin d'Études constitue une étape importante dans le parcours du Master en Data Science et Sécurité des Systèmes d'Information. Il permet de mobiliser les connaissances acquises durant la formation autour d'un projet concret, combinant analyse, conception, développement logiciel et validation technique.

Le présent projet, intitulé **Virtual FabLab**, s'inscrit dans le contexte de la transformation numérique des environnements de fabrication et d'apprentissage. Il vise à proposer une plateforme web permettant de visualiser un FabLab virtuel, de gérer ses machines, de simuler des opérations de fabrication numérique et d'exploiter des données IoT simulées pour la supervision intelligente.

Ce chapitre présente l'organisme d'accueil, le cadre général du projet, la problématique traitée ainsi que les objectifs poursuivis.

1.2 Présentation de l'ENSA Béni Mellal

L'École Nationale des Sciences Appliquées de Béni Mellal, désignée dans ce rapport par ENSA BM, est un établissement public d'enseignement supérieur relevant de l'Université Sultan Moulay Slimane. Elle participe à la formation de profils scientifiques et technologiques capables de répondre aux besoins du marché dans les domaines de l'ingénierie, de l'informatique, de la data science, des réseaux et des systèmes d'information.

L'ENSA BM offre un environnement académique favorable à l'apprentissage par projet, à l'innovation et à l'expérimentation technique. Dans ce cadre, les projets de fin d'études occupent une place essentielle, car ils permettent aux étudiants de confronter leurs compétences à des problématiques réelles ou proches du réel.

1.3 Missions et formations

1.3.1 Missions de l'établissement

Les missions principales de l'ENSA BM peuvent être résumées comme suit :

- assurer une formation scientifique et technique de haut niveau ;
- développer les compétences en ingénierie, en informatique et en technologies numériques ;
- encourager la recherche appliquée, l'innovation et l'esprit d'initiative ;

- préparer les étudiants à l'intégration professionnelle et à la conduite de projets complexes ;
- contribuer au développement régional et national à travers la formation de profils qualifiés.

1.3.2 Formations proposées

L'École Nationale des Sciences Appliquées de Béni Mellal propose des formations d'ingénieurs orientées vers les sciences appliquées, les technologies et l'ingénierie. Ces formations visent à développer des compétences scientifiques et techniques adaptées aux besoins du monde professionnel.

Les domaines de formation couvrent notamment :

- l'informatique et les technologies numériques ;
- les systèmes embarqués et connectés ;
- les télécommunications ;
- les systèmes industriels ;
- l'ingénierie et les sciences appliquées.

Dans ce contexte technologique, l'ENSA de Béni Mellal constitue un environnement adapté à la réalisation du projet Virtual FabLab, qui intègre le développement web, la simulation 3D, les technologies IoT et l'exploitation des données.

1.4 Organigramme de l'établissement

L'organisation de l'ENSA BM repose sur une structure administrative et pédagogique permettant d'assurer le suivi des formations, l'encadrement des étudiants et la gestion des activités académiques.

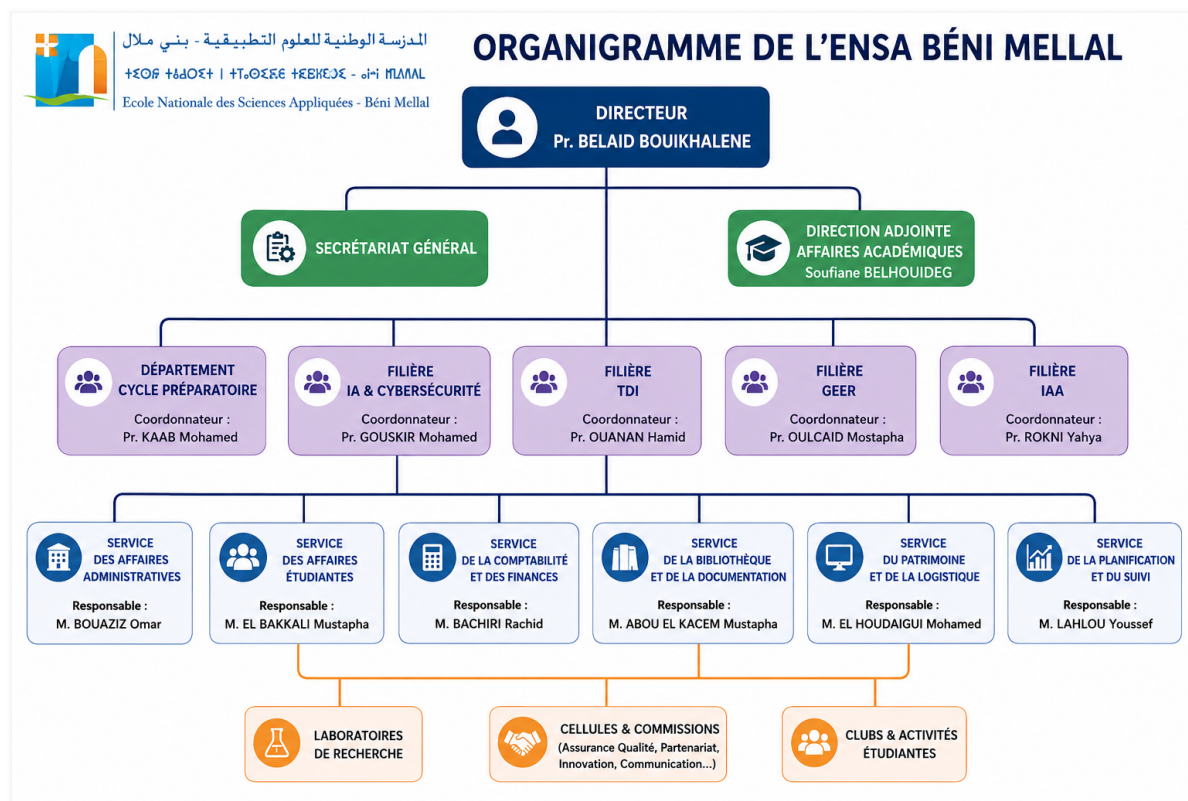


Figure 1.1: Organigramme de l'ENSA Béni Mellal

La figure 1.1 présente l'organisation générale de l'établissement. Cette structure permet de situer le projet dans son environnement académique et de comprendre le cadre institutionnel dans lequel il a été réalisé.

1.5 Contexte du stage

Le projet a été réalisé dans un contexte académique orienté vers l'innovation numérique et l'Industrie 4.0. Les FabLabs constituent aujourd'hui des espaces importants pour l'apprentissage pratique, le prototypage rapide et la fabrication numérique. Ils mettent à disposition des machines telles que les imprimantes 3D, les machines CNC ou les découpeuses laser.

Cependant, l'accès à ces équipements peut être limité par plusieurs contraintes : disponibilité des machines, besoin de présence physique, difficulté de supervision à distance et absence d'outils unifiés permettant de relier gestion, simulation et analyse de données.

Dans ce contexte, la création d'un FabLab virtuel représente une réponse pertinente. Elle permet de préparer les utilisateurs avant l'utilisation réelle des machines, de simuler des trajectoires de fabrication et d'introduire une couche de supervision intelligente fondée sur les données. Les données utilisées dans cette version sont simulées afin de valider l'architecture, les flux MQTT, le monitoring et le module d'analyse. La plateforme reste conçue pour être

connectée ultérieurement à des capteurs réels.

1.6 Problématique

Les FabLabs traditionnels reposent principalement sur une interaction physique avec les équipements. Cette approche est utile pour l'apprentissage pratique, mais elle présente certaines limites dans un contexte pédagogique moderne. Les étudiants ne disposent pas toujours d'un moyen simple pour visualiser les machines, tester un fichier de fabrication ou consulter l'état des équipements avant une séance.

Par ailleurs, les machines de fabrication numérique produisent ou pourraient produire des données de fonctionnement telles que la température, la vibration, la durée d'utilisation, la vitesse du moteur ou les erreurs signalées. Ces données sont précieuses pour la supervision et la maintenance, mais elles restent souvent peu exploitées dans un cadre académique. Dans la version réalisée, ces mesures ne proviennent pas encore de capteurs physiques : elles sont générées par un simulateur MQTT afin de tester les traitements et les interfaces dans des conditions contrôlées.

La problématique du projet peut donc être formulée ainsi :

Comment concevoir et réaliser une plateforme virtuelle permettant de gérer un FabLab, de simuler ses machines et d'exploiter des données IoT simulées pour la supervision intelligente et la maintenance prédictive ?

1.7 Objectifs du projet

L'objectif principal du projet est de concevoir et développer une plateforme web nommée **Virtual FabLab**, intégrant la gestion des utilisateurs et des machines, la visualisation 3D, la simulation de fichiers de fabrication et l'exploitation de données IoT simulées.

Les objectifs spécifiques sont les suivants :

- mettre en place une interface web moderne destinée aux étudiants et aux administrateurs ;
- permettre l'inscription, l'authentification et la gestion des profils utilisateurs ;
- gérer les machines du FabLab, leurs types, leurs états et leur positionnement dans l'espace 3D ;
- offrir un système de réservation des machines avec validation administrative ;
- construire une scène 3D interactive représentant le FabLab et ses équipements ;

- simuler le comportement d'une imprimante 3D et d'une machine CNC à partir de fichiers G-code ou NC ;
- intégrer une télémétrie simulée transmise via MQTT ;
- analyser les données simulées afin de détecter les anomalies et d'estimer l'état de santé des machines ;
- classifier les intentions administrateur liées à la supervision et à la maintenance ;
- proposer un Copilote IA d'aide à la décision réservé à l'administrateur ;
- afficher les indicateurs de supervision dans un tableau de bord administrateur.

1.8 Présentation générale du Virtual FabLab

La plateforme Virtual FabLab est conçue comme une application web complète reposant sur une architecture frontend/backend. Le frontend assure l'expérience utilisateur, la visualisation du laboratoire virtuel, les interfaces de réservation, les écrans administrateur et les vues de simulation. Le backend fournit les API, la persistance des données, l'authentification, la gestion des rôles, la réception des données MQTT et les services d'analyse.

La solution réalisée couvre plusieurs modules complémentaires :

- un module d'authentification basé sur des jetons JWT ;
- un module de gestion des utilisateurs et des droits d'accès ;
- un module de gestion des machines et de leurs emplacements dans le FabLab virtuel ;
- un module de réservation avec suivi des demandes ;
- un module de visualisation 3D du laboratoire ;
- un module de simulation G-code pour imprimante 3D et CNC ;
- un module MQTT permettant l'ingestion de télémétrie simulée ;
- un module Data Science et IA pour la détection d'anomalies, le score de santé, l'estimation indicative du risque de maintenance, la classification d'intentions et le Copilote IA d'aide à la décision.

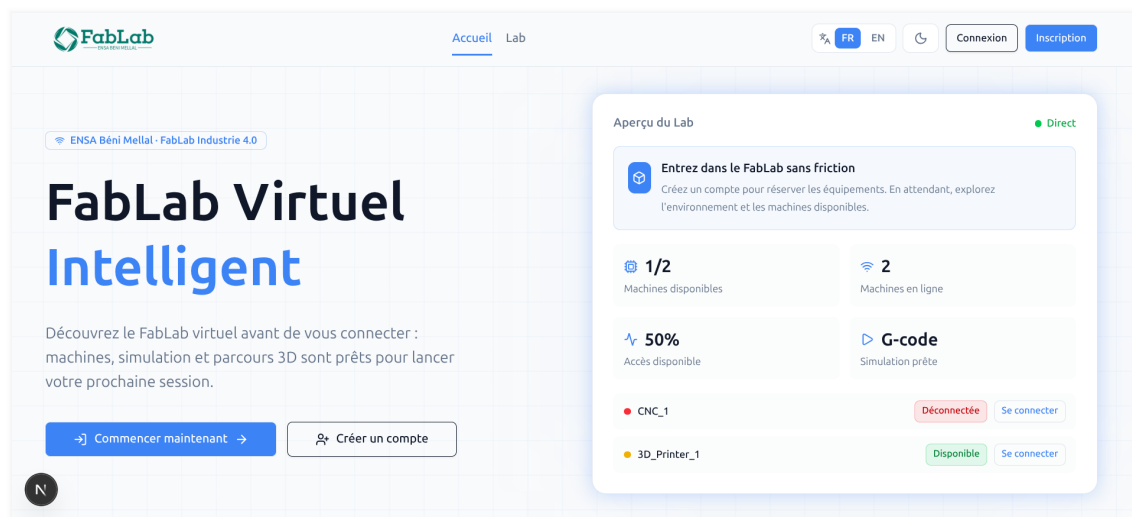


Figure 1.2: Vue générale prévue de la plateforme Virtual FabLab

1.9 Conclusion

Ce chapitre a présenté l'organisme d'accueil, le contexte général du projet, la problématique ainsi que les objectifs de la plateforme Virtual FabLab. Le projet se positionne à l'intersection du développement web, de la simulation 3D, de l'IoT et de la Data Science.

Le chapitre suivant sera consacré à l'analyse des besoins et à la conception du système. Il présentera les acteurs, les fonctionnalités attendues, l'architecture générale, les modules de la plateforme ainsi que les technologies retenues pour la réalisation.

CHAPITRE 2

ANALYSE DES BESOINS ET CONCEPTION

2.1 Introduction

Après avoir présenté le cadre général du projet, ce chapitre détaille l’analyse des besoins et la conception de la plateforme Virtual FabLab. L’objectif est de formaliser les fonctionnalités attendues, les contraintes de qualité, les acteurs du système, puis de proposer une architecture technique cohérente avec l’implémentation réalisée.

La conception retenue s’appuie sur une application web composée d’un frontend interactif, d’un backend applicatif, d’une base de données, d’un flux MQTT simulé et d’un module d’analyse de données.

2.2 Étude des besoins

L’étude des besoins vise à identifier les services que la plateforme doit fournir aux utilisateurs. Le système doit répondre à deux catégories principales d’utilisateurs : les étudiants et les administrateurs.

L’étudiant doit pouvoir accéder au FabLab virtuel, consulter les machines disponibles, réserver un créneau et tester un fichier de fabrication dans une simulation 3D. L’administrateur doit disposer d’un espace de pilotage lui permettant de gérer les machines, les utilisateurs, les réservations et les indicateurs de supervision.

Le système doit également être capable de recevoir des données simulées issues des machines, de les stocker et de les exploiter pour la détection d’anomalies et l’aide à la maintenance. Les données utilisées dans cette version sont simulées afin de valider l’architecture, les flux MQTT, le monitoring et le module d’analyse. La plateforme reste conçue pour être connectée ultérieurement à des capteurs réels.

2.3 Besoins fonctionnels

2.3.1 Besoins de l’étudiant

Les fonctionnalités attendues pour l’étudiant sont les suivantes :

- créer un compte utilisateur ;
- s’authentifier et accéder à un espace personnel ;
- consulter les machines disponibles dans le FabLab ;
- visualiser le laboratoire virtuel en 3D ;
- sélectionner une machine et consulter ses informations ;

- effectuer une demande de réservation sur un créneau disponible ;
- consulter l'historique de ses réservations ;
- importer un fichier G-code ou NC compatible ;
- lancer, mettre en pause et réinitialiser une simulation ;
- modifier les informations de son profil.

2.3.2 Besoins de l'administrateur

Les fonctionnalités attendues pour l'administrateur sont les suivantes :

- consulter un tableau de bord global ;
- gérer les utilisateurs et leurs rôles ;
- activer ou désactiver un compte utilisateur ;
- créer, modifier ou supprimer des machines ;
- modifier l'état et le placement 3D des machines ;
- consulter les demandes de réservation ;
- approuver, refuser ou supprimer une réservation ;
- superviser les données de télémétrie reçues ;
- suivre l'état MQTT du système ;
- consulter les alertes d'anomalies ;
- consulter les scores de santé et les recommandations de maintenance ;
- interroger un Copilote IA de maintenance pour obtenir une synthèse structurée fondée sur les données disponibles.

2.4 Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités attendues de la solution :

- **Sécurité** : les fonctionnalités sensibles doivent être protégées par authentification et contrôle de rôle.
- **Ergonomie** : l'interface doit être claire, lisible et adaptée à un usage pédagogique.
- **Maintenabilité** : le code doit être organisé en modules séparant les responsabilités.
- **Évolutivité** : la solution doit permettre l'ajout de nouveaux types de machines ou de nouveaux indicateurs.

- **Performance** : la scène 3D et la simulation doivent rester fluides dans un navigateur moderne.
- **Traçabilité** : les données de télémétrie doivent être conservées afin d'alimenter l'historique et les analyses.
- **Interopérabilité** : les échanges entre frontend et backend doivent être réalisés à travers des API structurées.

2.5 Acteurs du système

Le système met en interaction plusieurs acteurs :

- **Étudiant** : utilisateur inscrit qui consulte les machines, réserve des créneaux et lance des simulations.
- **Administrateur** : utilisateur disposant de droits avancés pour gérer la plateforme et superviser les machines.
- **Broker MQTT** : composant intermédiaire chargé de transmettre les messages de télémétrie simulée.
- **Machine simulée** : source logique des données de fonctionnement publiées dans le système.

2.6 Cas d'utilisation

Le diagramme de cas d'utilisation permet de représenter les interactions principales entre les acteurs et la plateforme.

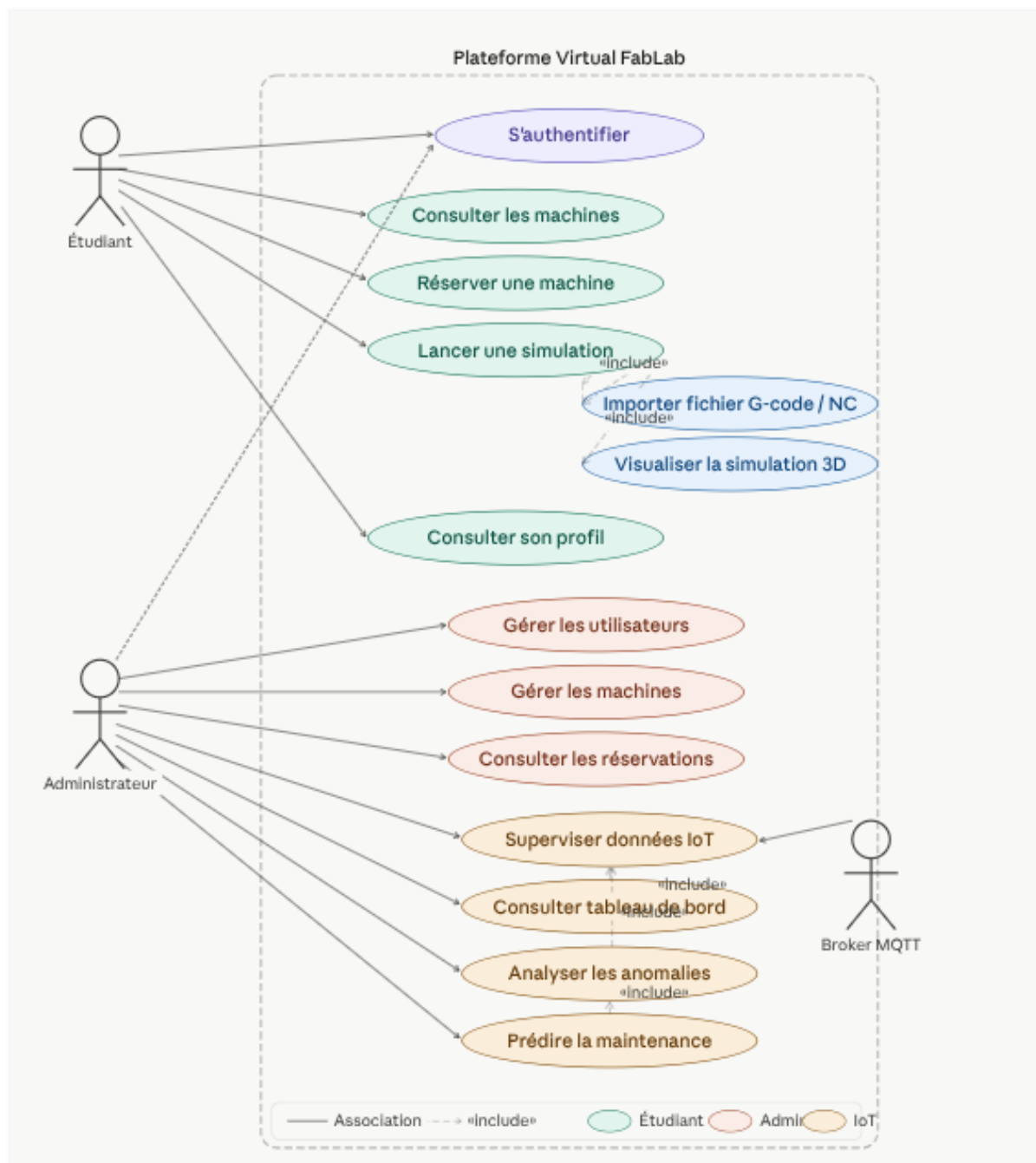


Figure 2.1: Diagramme de cas d'utilisation de la plateforme Virtual FabLab

Les cas d'utilisation principaux sont l'authentification, la consultation des machines, la réservation, la simulation, la gestion administrative, la supervision IoT et l'analyse des anomalies.

2.7 Architecture générale

L'architecture générale de la plateforme repose sur une séparation claire entre l'interface utilisateur, les services backend, la base de données, le flux MQTT et le module Data Science.

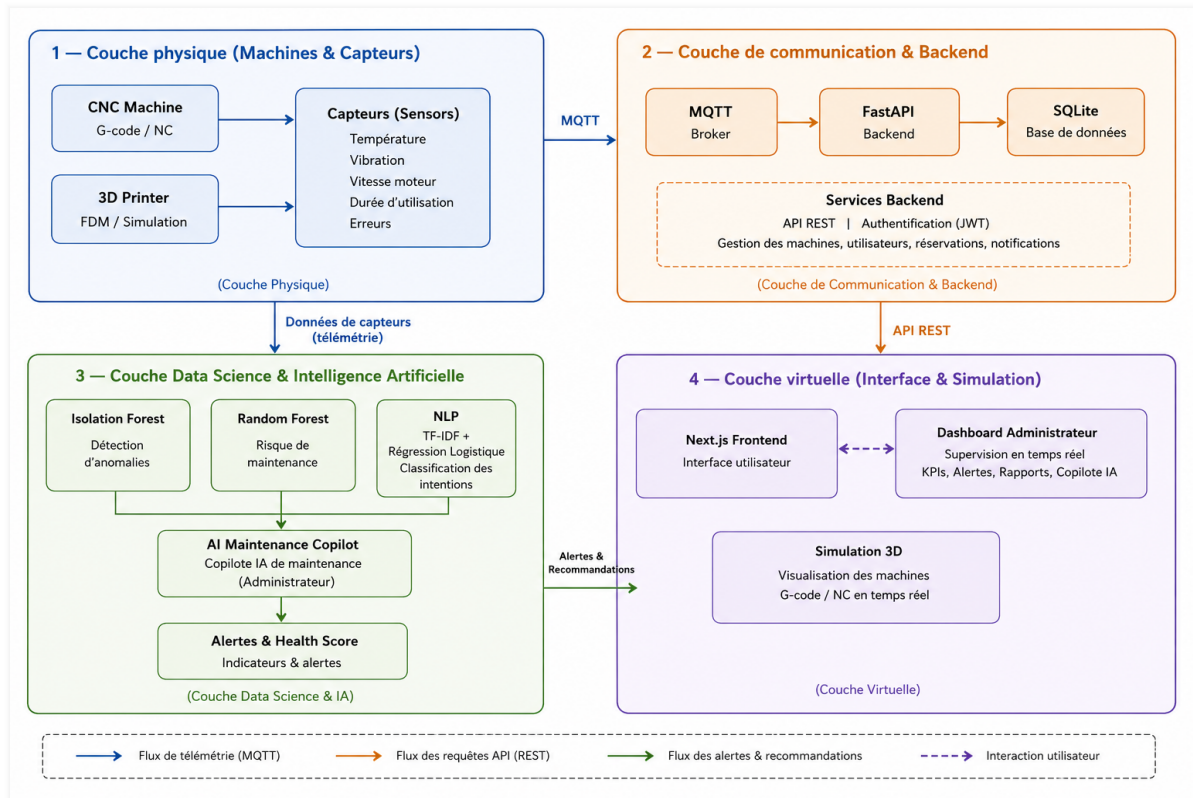


Figure 2.2: Architecture générale de la plateforme Virtual FabLab

Le frontend communique avec le backend à travers des requêtes HTTP. Le backend expose les API, gère la base de données SQLite et lance un abonné MQTT au démarrage de l'application. Les messages reçus sont transformés en mesures de télémétrie puis exploités par les services de monitoring et de prédiction.

2.8 Description des modules

La plateforme est structurée autour des modules suivants :

- **Module d'authentification** : inscription, connexion, profil, changement de mot de passe et génération de jetons JWT.
- **Module utilisateurs** : liste, recherche, changement de rôle et activation ou désactivation des comptes.
- **Module machines** : gestion des types de machines, des instances, de leurs états et de leurs coordonnées dans le laboratoire 3D.
- **Module réservations** : création des demandes, calcul des créneaux disponibles, annulation et validation par l'administrateur.
- **Module FabLab 3D** : visualisation interactive de la salle et des machines avec

sélection et consultation de leur état.

- **Module simulation** : importation et interprétation des fichiers G-code ou NC pour l'imprimante 3D et la CNC.
- **Module MQTT** : réception des données simulées publiées sur les topics machines.
- **Module Data Science et IA** : détection d'anomalies avec Isolation Forest, estimation du risque de maintenance avec Random Forest Classifier, calcul du score de santé, classification supervisée des intentions administrateur et Copilote IA d'aide à la décision.
- **Module notifications** : notification des étudiants après validation ou refus d'une réservation.

2.9 Diagramme de classes

Le diagramme de classes représente les entités principales utilisées dans la base de données et leurs relations : utilisateur, machine, type de machine, réservation, mesure capteur et notification.

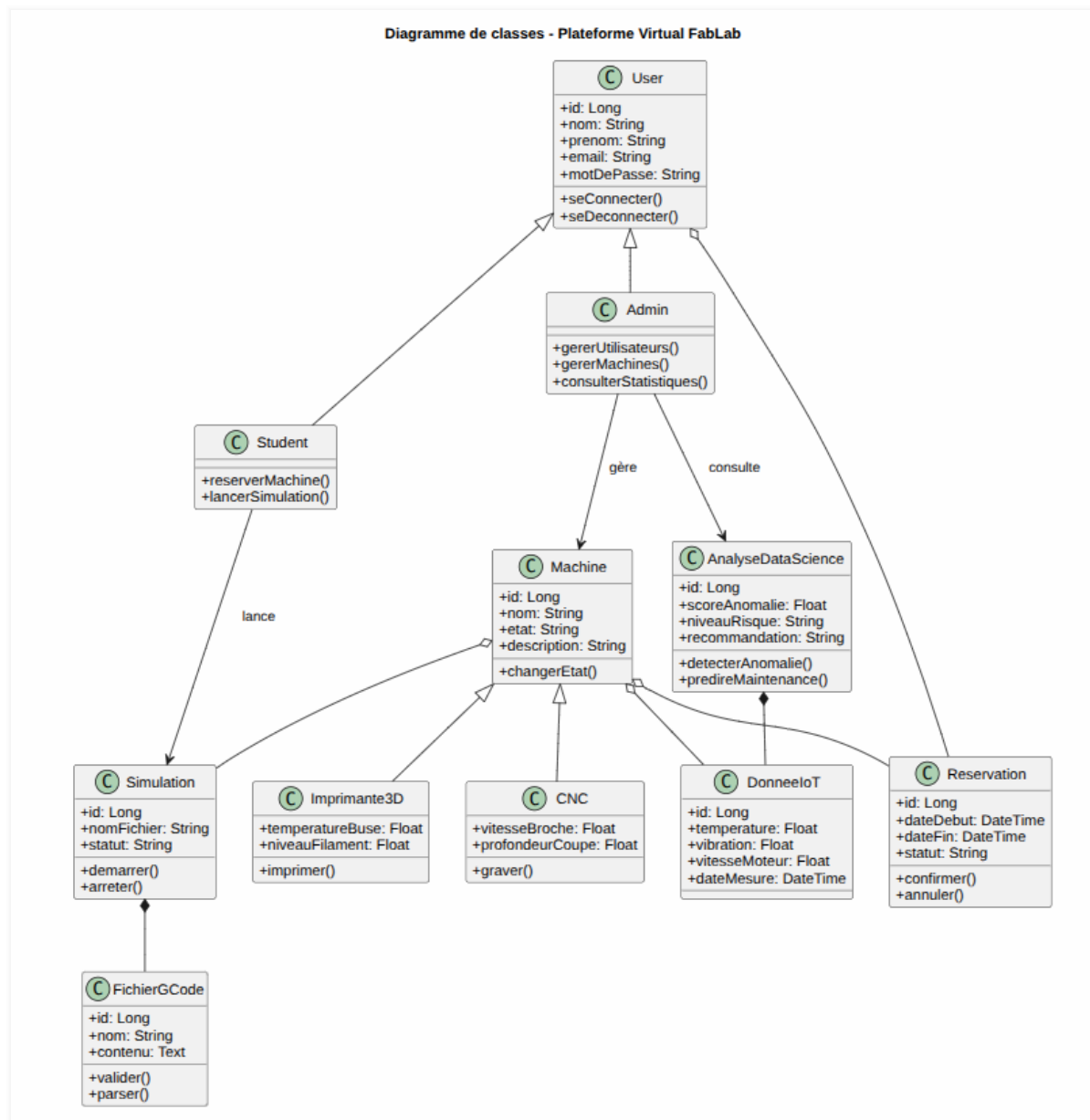


Figure 2.3: Diagramme de classes de la plateforme Virtual FabLab

La conception des données distingue les types de machines des instances de machines. Cette séparation permet d'associer un modèle 3D, un schéma de capteurs et des informations techniques à un type, tout en laissant chaque instance posséder son propre nom, état et emplacement.

2.10 Conception du système

2.10.1 Conception de l'authentification

Le système d'authentification repose sur des jetons JWT. Après connexion, le backend renvoie un jeton d'accès utilisé par le frontend pour appeler les routes protégées. Le rôle de

l'utilisateur permet de distinguer les droits d'un étudiant et ceux d'un administrateur.

2.10.2 Conception de la gestion des machines

Chaque machine possède un type, un nom unique, un état, des notes et des paramètres de placement dans la scène 3D. Les machines peuvent être consultées par les utilisateurs, tandis que leur création, modification ou suppression est réservée à l'administrateur.

2.10.3 Conception de la réservation

Le système de réservation repose sur des créneaux d'une heure entre 8h00 et 20h00. Une demande est créée avec l'état **pending**. L'administrateur peut ensuite l'approuver ou la refuser. Les conflits sont évités en vérifiant les réservations actives pour la même machine et le même intervalle.

2.10.4 Conception de la télémétrie

Dans l'implémentation actuelle, la télémétrie simulée est reçue sous forme de messages JSON publiés via MQTT. Le backend associe chaque message à une machine existante, enregistre les mesures et expose l'historique à travers les routes de monitoring.

2.10.5 Conception de l'analyse intelligente

Le module Data Science et IA combine des modèles entraînés avec scikit-learn et une logique par règles. La détection d'anomalies s'appuie sur un modèle *Isolation Forest*, tandis que l'estimation du risque de maintenance repose sur un *Random Forest Classifier*. Une logique complémentaire par règles est utilisée pour consolider le score de santé, le niveau de risque et les recommandations de maintenance, notamment lorsque les modèles ne sont pas disponibles ou lorsqu'une lecture doit être interprétée de manière explicable.

Les variables analysées sont la température, la vibration, la durée d'utilisation exprimée sous forme `usage_duration` puis convertie en `usage_hours`, la vitesse du moteur `motor_speed` et l'historique des erreurs signalées. Les sorties produites par le module sont le statut d'anomalie, la probabilité indicative de risque, le score de santé, le niveau de risque et une recommandation de maintenance.

Le même module intègre également une classification supervisée des intentions administrateur, fondée sur TF-IDF et Logistic Regression, afin d'alimenter un Copilote IA de maintenance. Ce copilote ne correspond pas à un LLM ni à une IA générative ; il fournit des réponses déterministes construites à partir des données de la plateforme, des résultats analytiques et de l'intention reconnue.

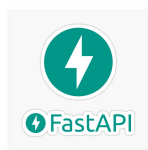
2.11 Technologies utilisées

2.11.1 Next.js



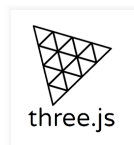
Next.js est un framework basé sur React permettant de développer des applications web modernes, modulaires et performantes. Dans ce projet, il constitue la base du frontend. Il organise les pages de l'application, les composants d'interface, les appels API et les vues interactives destinées aux étudiants et aux administrateurs.

2.11.2 FastAPI



FastAPI est un framework Python destiné à la création d'API web rapides et structurées. Il est utilisé pour implémenter le backend de la plateforme : authentification, gestion des machines, réservations, monitoring, réception MQTT et services d'analyse.

2.11.3 Three.js



Three.js est une bibliothèque JavaScript permettant de créer des scènes 3D dans le navigateur. Elle est utilisée pour afficher le laboratoire virtuel, charger les modèles GLB des machines et animer les trajectoires issues des fichiers de fabrication.

2.11.4 Blender



Blender est un logiciel de modélisation 3D open source. Il intervient dans la préparation et l'optimisation des modèles 3D utilisés dans la plateforme, notamment ceux de l'imprimante 3D et de la machine CNC.

2.11.5 MQTT



MQTT est un protocole léger de communication publish/subscribe très utilisé dans les systèmes IoT. Dans ce projet, il permet de simuler l'envoi de mesures machines vers le backend à l'aide d'un simulateur MQTT. Les messages sont publiés sur des topics propres à chaque machine, selon le motif suivant :

```
fablab/machines/{machine_id}/data
```

2.11.6 Python



Python est utilisé côté backend pour développer les services applicatifs et les traitements de données. Il facilite également l'intégration des bibliothèques de Data Science et l'entraînement des modèles.

2.11.7 Tailwind CSS



Tailwind CSS est un framework CSS utilitaire permettant de construire rapidement des interfaces cohérentes et responsives. Dans la plateforme, il est utilisé pour structurer les pages, les cartes, les tableaux, les formulaires et les tableaux de bord.

2.11.8 Scikit-learn



Scikit-learn est une bibliothèque Python dédiée au Machine Learning. Elle est utilisée dans ce projet pour entraîner et exploiter un modèle de détection d'anomalies de type Isolation Forest et un Random Forest Classifier pour l'estimation du risque de maintenance.

2.11.9 Visual Studio Code



Visual Studio Code est l'environnement de développement utilisé pour organiser et modifier les différents modules du projet : frontend, backend, simulation, scripts Python et fichiers LaTeX du rapport.

2.11.10 Technologies et équipements IoT

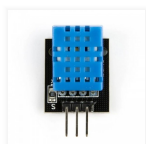
Cette partie présente les composants matériels et les équipements de fabrication numérique liés à la dimension IoT du FabLab. Dans la version actuelle, ils servent de référence pour concevoir les mesures et les scénarios de supervision. La collecte effective n'est pas encore réalisée par des capteurs physiques : elle est simulée par MQTT afin de valider les flux, le stockage et l'analyse avant une intégration matérielle future.

ESP8266



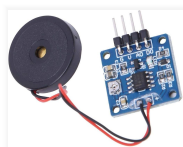
L'ESP8266 est un microcontrôleur doté d'une connectivité Wi-Fi intégrée, largement utilisé dans les applications de l'Internet des Objets. Dans un contexte FabLab, il peut servir d'interface entre les capteurs physiques et la plateforme de supervision. Il permet de collecter des mesures, de les traiter localement puis de les transmettre vers un serveur ou un broker MQTT. Son faible coût et sa facilité de programmation en font un choix adapté aux prototypes académiques et industriels légers.

Capteur de température



Le capteur de température permet de mesurer l'échauffement d'un environnement, d'un composant électronique ou d'une machine en fonctionnement. Dans le cadre d'un FabLab intelligent, cette donnée est utile pour surveiller les conditions de travail et prévenir les situations anormales. Une température excessive peut indiquer une surcharge, un défaut de ventilation ou une dégradation progressive d'un équipement. Lors d'une connexion future à des capteurs réels, ces mesures pourront alimenter des tableaux de bord et des modèles de maintenance prédictive.

Capteur de vibration



Le capteur de vibration mesure les oscillations mécaniques produites par une machine pendant son fonctionnement. Ces données sont essentielles pour les équipements de fabrication numérique, car des vibrations inhabituelles peuvent révéler un désalignement, une usure ou un défaut mécanique. Dans une approche de maintenance préventive, l'analyse de ces signaux aide à identifier les anomalies avant l'apparition d'une panne. Les données de vibration constituent ainsi un indicateur pertinent de l'état de santé des machines.

Imprimante 3D



L'imprimante 3D est un équipement de fabrication additive permettant de produire des objets physiques à partir de modèles numériques. Elle fonctionne généralement par dépôt progressif de matière selon des trajectoires définies dans un fichier G-code. Dans un FabLab, elle constitue un outil pédagogique et technique essentiel pour le prototypage rapide. Son intégration à une plateforme IoT permet de suivre son état, sa température, sa progression et ses éventuelles anomalies de fonctionnement.

Machine CNC



La machine CNC est un équipement de fabrication soustractive commandé numériquement. Elle exécute des opérations d'usinage à partir d'instructions précises, souvent décrites dans des fichiers NC ou G-code. Dans un FabLab, elle permet de travailler différents matériaux avec une grande précision. La supervision IoT d'une CNC peut porter sur son état, ses vibrations, sa durée d'utilisation et ses paramètres de fonctionnement afin d'améliorer la sécurité, la traçabilité et la maintenance.

2.12 Conclusion

Ce chapitre a présenté l'analyse des besoins, les acteurs, les cas d'utilisation et la conception générale de la plateforme. Il a également décrit les technologies retenues et leur rôle dans le projet.

Le chapitre suivant sera consacré à la réalisation et à l'implémentation. Il détaillera l'organisation concrète du frontend et du backend, les modules développés, la simulation 3D, le flux MQTT et les interfaces de supervision.

CHAPITRE 3

RÉALISATION ET IMPLÉMENTATION

3.1 Introduction

Ce chapitre présente la réalisation technique de la plateforme Virtual FabLab. Contrairement au chapitre précédent, qui décrit l'analyse et la conception, cette partie se concentre sur l'implémentation effective du projet : organisation du code, architecture frontend/backend, modules fonctionnels, simulation 3D, communication MQTT et tableau de bord de supervision.

La plateforme réalisée repose sur un backend FastAPI, une interface Next.js et des modules spécialisés pour la visualisation 3D, l'interprétation des fichiers de fabrication et l'analyse des données de télémétrie. Les données utilisées dans cette version sont simulées afin de valider l'architecture, les flux MQTT, le monitoring et le module d'analyse. La plateforme reste conçue pour être connectée ultérieurement à des capteurs réels.

3.2 Architecture frontend/backend

L'implémentation adopte une architecture séparant clairement le frontend et le backend. Le frontend prend en charge l'interface utilisateur, la navigation, les scènes 3D et les tableaux de bord. Le backend fournit les API, la sécurité, la base de données, l'ingestion MQTT et les traitements analytiques.

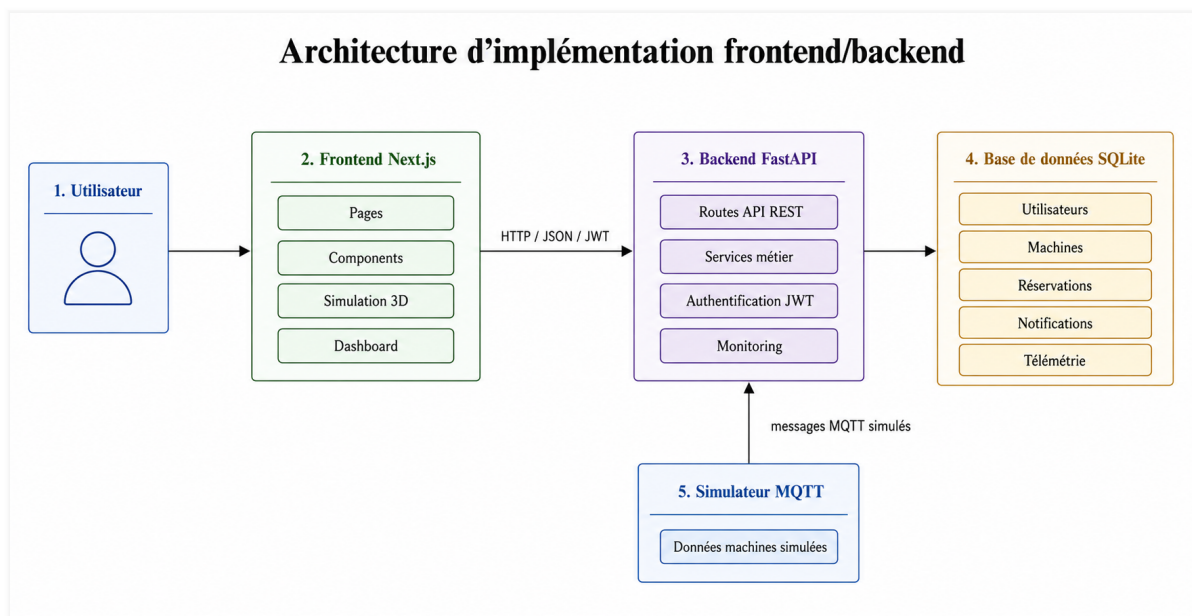


Figure 3.1: Architecture d'implémentation frontend/backend

Le backend est initialisé dans le point d'entrée FastAPI. Au démarrage, il initialise la base de données et lance le service abonné MQTT. Les routes sont ensuite organisées par

domaine fonctionnel : authentification, machines, réservations, administration, monitoring, simulation, notifications et intelligence artificielle.

3.3 Organisation des dossiers

Le projet est organisé en trois parties principales : le frontend, le backend et le rapport LaTeX. Cette organisation facilite la maintenance et permet d'isoler les responsabilités.

Table 3.1: *Organisation générale du projet*

Dossier	Rôle
Frontend/app	Pages principales de l'application Next.js : accueil, lab 3D, simulation, dashboard, réservations, utilisateurs et profil.
Frontend/components	Composants réutilisables : layout, authentification, scène 3D, simulation, tableaux de bord et composants UI.
Frontend/lib	Fonctions utilitaires, appels API, traductions et moteur d'interprétation des fichiers G-code/NC.
backend/app/routes	Définition des endpoints FastAPI par domaine fonctionnel.
backend/app/services	Logique métier : authentification, machines, réservations, télémétrie, MQTT, anomalies et prédiction.
backend/app/models	Modèles SQLAlchemy représentant les entités persistées.
backend/scripts	Scripts de simulation MQTT et d'entraînement des modèles de Machine Learning.
Rapport_PFE	Sources LaTeX du rapport académique.

3.4 Implémentation du backend

3.4.1 API FastAPI

Le backend expose une API REST structurée autour de plusieurs routeurs. Chaque routeur regroupe les endpoints liés à un domaine précis. Cette organisation améliore la lisibilité du code et facilite l'évolution de la plateforme.

Les principales familles d'endpoints sont :

- `/auth` pour l'inscription, la connexion et le profil ;
- `/machines` pour la gestion administrative des machines ;
- `/lab/machines` pour l'affichage public des machines du FabLab ;
- `/reservations` pour les demandes de réservation ;

- /admin pour la gestion des utilisateurs et des réservations ;
- /monitoring pour la télémétrie et l'état MQTT ;
- /simulation pour l'état de simulation ;
- /admin/ai pour les analyses intelligentes réservées à l'administrateur.

3.4.2 Base de données

La persistance est assurée par SQLite avec SQLAlchemy. Les modèles principaux sont les utilisateurs, les types de machines, les machines, les réservations, les mesures capteurs et les notifications.

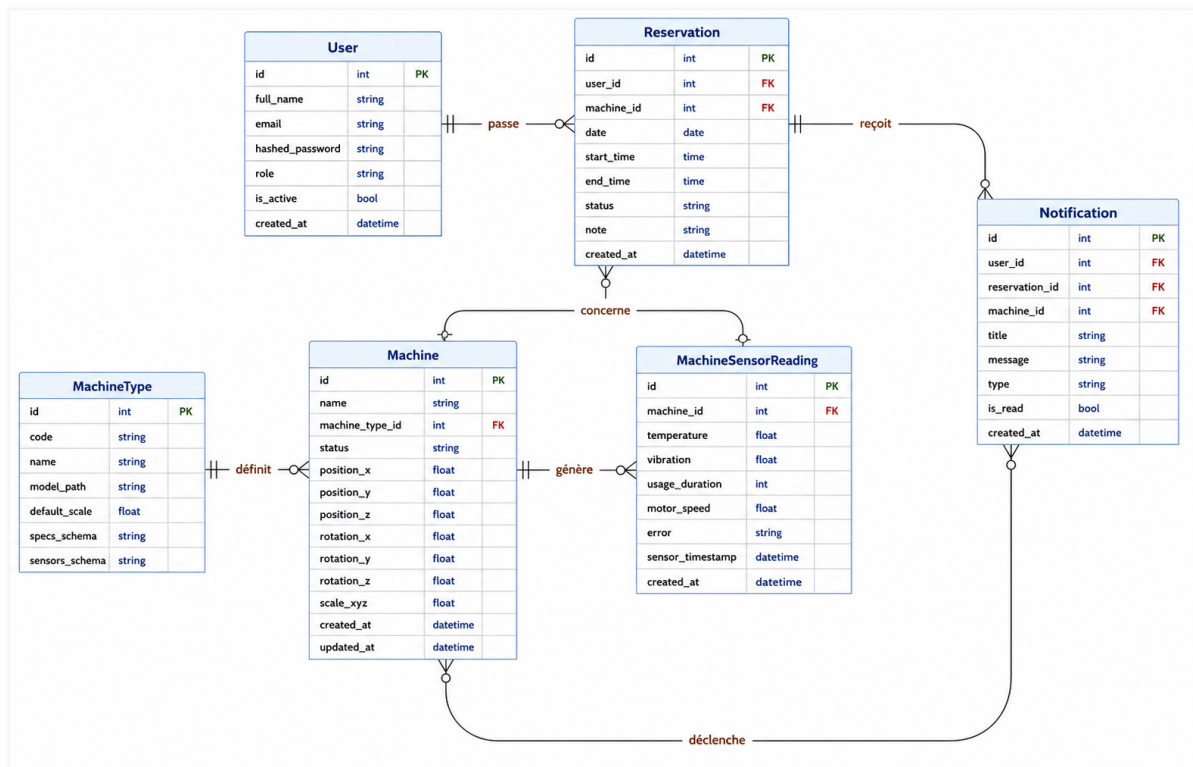


Figure 3.2: Schéma logique de la base de données

3.5 Système d'authentification

L'authentification est basée sur des jetons JWT. Lorsqu'un utilisateur se connecte, le backend vérifie son email et son mot de passe hashé, puis génère un jeton d'accès. Le frontend conserve ce jeton et l'ajoute dans l'en-tête **Authorization** lors des appels aux routes protégées.

Deux rôles sont définis dans le système :

- **student** : rôle par défaut pour les utilisateurs inscrits ;
- **admin** : rôle permettant l'accès aux interfaces et endpoints d'administration.

3.6 Gestion des utilisateurs et de l'administration

L'espace administrateur permet de consulter les utilisateurs, de rechercher un compte, de filtrer par rôle ou par état, puis de modifier les droits. L'administrateur peut également activer ou désactiver un compte.

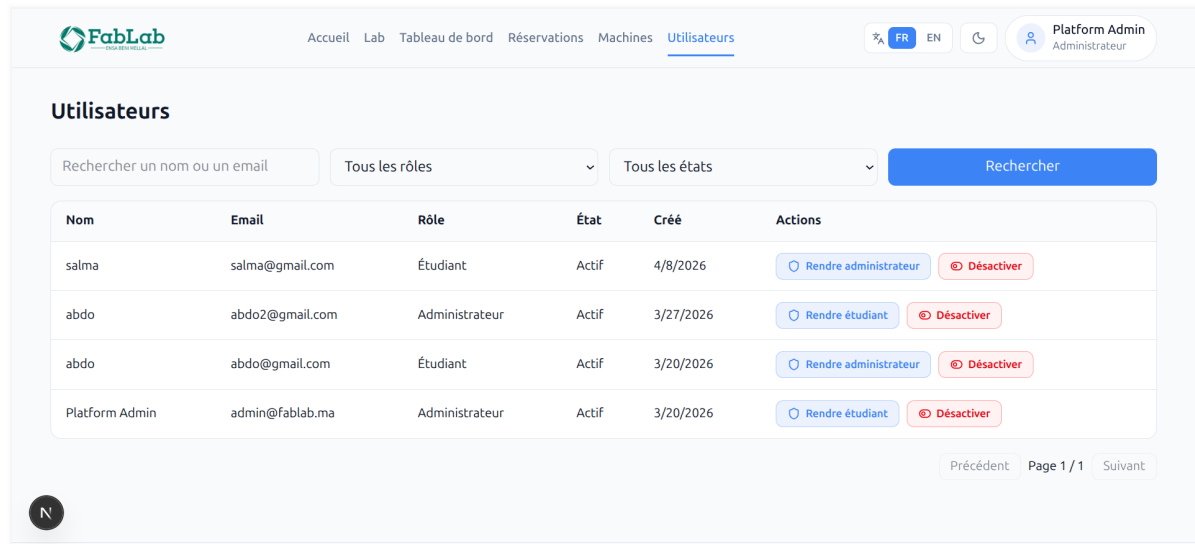


Figure 3.3: Interface de gestion des utilisateurs

L'accès à ces fonctionnalités est protégé par une dépendance backend qui vérifie que l'utilisateur courant possède le rôle administrateur.

3.7 Gestion des machines

Le module machines permet de gérer les instances du FabLab. Chaque machine possède un type, un nom, un état, des notes et des paramètres de placement dans la scène 3D : position, rotation et échelle.

Les types de machines initialisés par défaut incluent notamment :

- une imprimante 3D associée au modèle `/models/3d_printer.glb` ;
- une machine CNC associée au modèle `/models/CNC.glb` ;

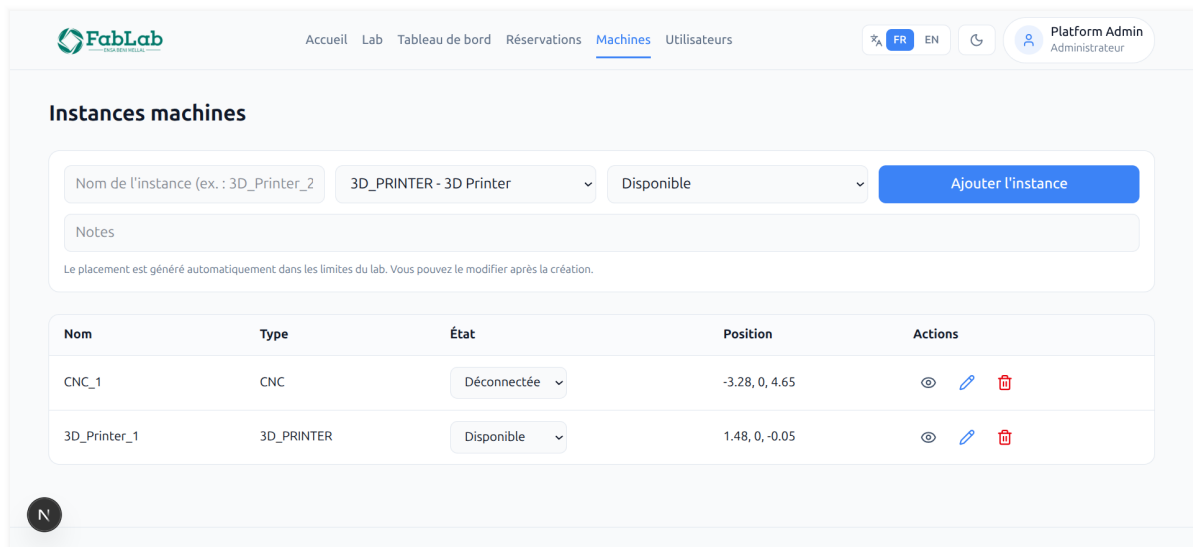


Figure 3.4: Interface de gestion des machines

3.8 Système de réservation

Le système de réservation permet à l'étudiant de choisir une machine, une date et un créneau horaire. Les créneaux sont générés entre 8h00 et 20h00 avec une durée d'une heure. Le backend vérifie les conflits afin d'empêcher deux réservations actives sur la même machine et le même intervalle.

Une demande de réservation est créée avec l'état **pending**. L'administrateur peut ensuite approuver ou refuser cette demande. Lorsqu'une décision est prise, une notification est créée pour l'utilisateur concerné.

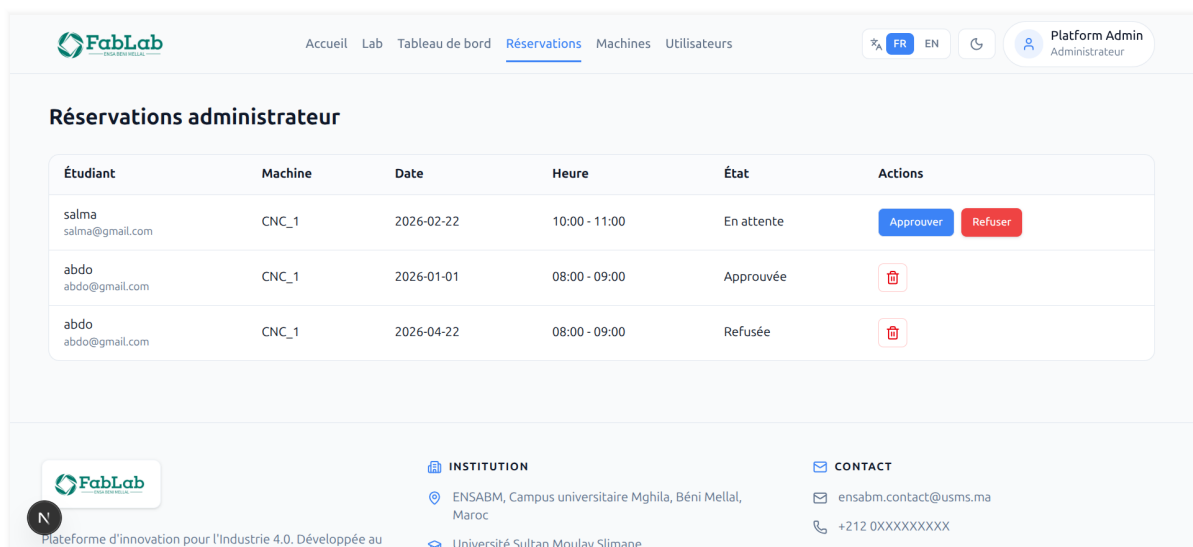


Figure 3.5: Interface de réservation des machines

3.9 FabLab virtuel en 3D

La page du laboratoire virtuel affiche une scène 3D représentant l'espace du FabLab. Les machines sont chargées dynamiquement depuis le backend, ce qui permet d'afficher les machines réellement enregistrées dans la base de données.

Chaque machine possède un modèle GLB, une position, une rotation et une échelle. L'utilisateur peut sélectionner une machine dans la scène afin de consulter son état, ses mesures récentes et ses actions disponibles : consulter les détails, ouvrir la simulation ou réserver.

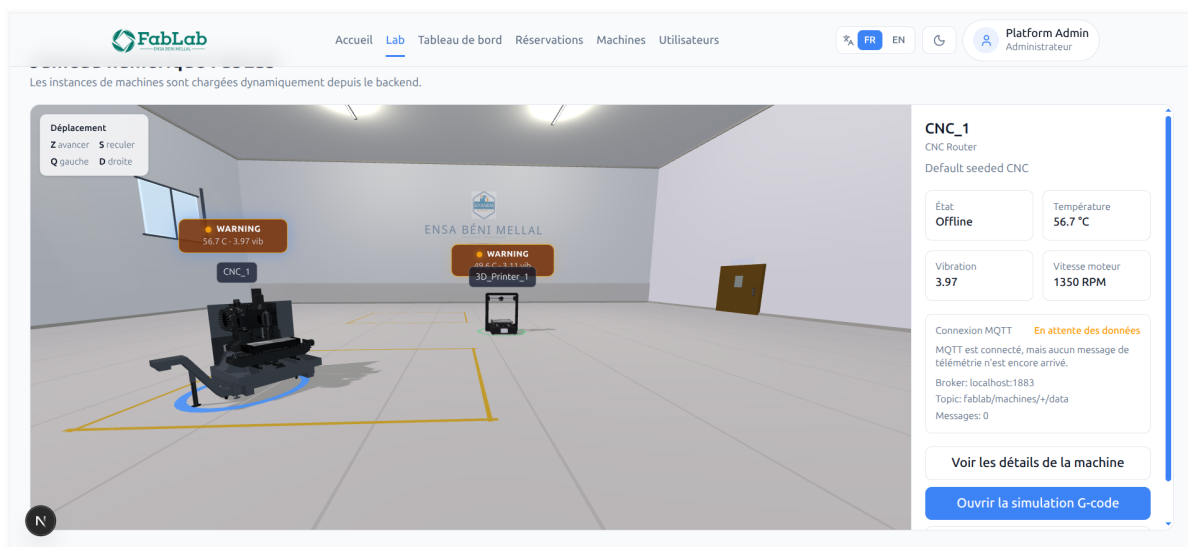


Figure 3.6: Vue 3D du FabLab virtuel

3.10 Simulation CNC

La simulation CNC repose sur l'interprétation de fichiers NC ou G-code adaptés à une machine de fraisage ou de découpe. Le module détecte les commandes de mouvement, les vitesses d'avance, les arcs, les commandes de broche et certains signaux caractéristiques de la CNC.

Les trajectoires sont transformées en mouvements simulables dans la scène 3D. Les déplacements rapides et les opérations de coupe sont représentés différemment afin de distinguer les phases de positionnement et les phases d'usinage.

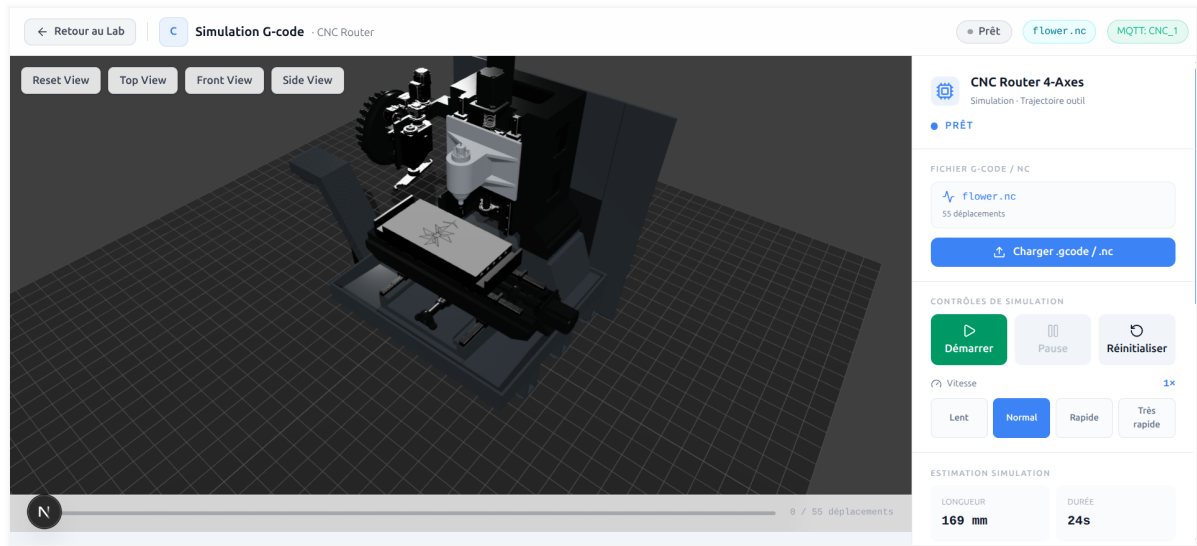


Figure 3.7: Simulation d'une trajectoire CNC

3.11 Simulation de l'imprimante 3D

La simulation de l'imprimante 3D prend en charge les fichiers G-code contenant les déplacements, l'axe d'extrusion, les vitesses d'avance et certaines commandes propres à l'impression 3D. Le système distingue les déplacements sans extrusion des segments imprimés.

La scène affiche progressivement la trajectoire d'impression et peut représenter les couches de l'objet. Le panneau latéral permet de suivre la progression, la position courante, la vitesse de lecture, la température simulée et les informations associées à la télémétrie lorsque la machine sélectionnée est liée à une instance enregistrée dans le backend.

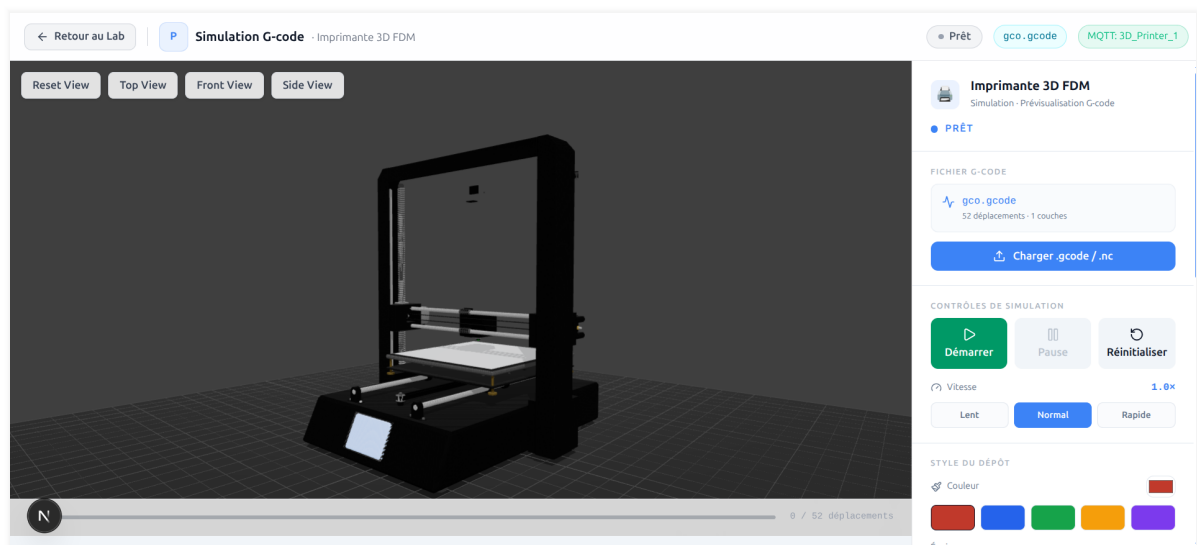


Figure 3.8: Simulation d'un fichier G-code pour imprimante 3D

3.12 Interprétation des fichiers G-code et NC

L'interprétation des fichiers est réalisée côté frontend afin de fournir un retour immédiat à l'utilisateur. Les lignes sont normalisées, les commentaires sont ignorés et les tokens de type G, M, X, Y, Z, F, E, I, J ou R sont analysés.

Le système vérifie également la compatibilité du fichier avec la machine sélectionnée. Par exemple, un fichier contenant des commandes de broche ou de refroidissement est considéré comme non compatible avec l'imprimante 3D, tandis qu'un fichier contenant des commandes d'extrusion est rejeté pour la CNC.

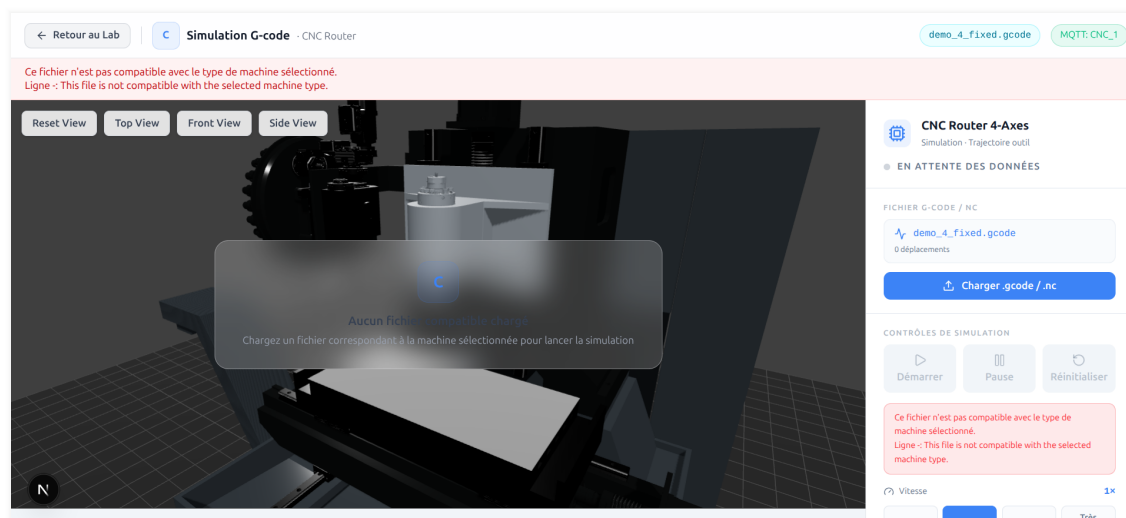


Figure 3.9: Processus d'interprétation et de validation des fichiers G-code/NC

3.13 Télémétrie simulée avec MQTT

La télémétrie est simulée par un script Python qui publie périodiquement des messages MQTT. Ce choix permet de valider le flux complet sans dépendre, à ce stade, d'une installation physique de capteurs. Les données utilisées dans cette version sont simulées afin de valider l'architecture, les flux MQTT, le monitoring et le module d'analyse. La plateforme reste conçue pour être connectée ultérieurement à des capteurs réels. Chaque message est envoyé sur un topic de la forme :

`fablab/machines/{machine_id}/data`

Le message contient l'identifiant de la machine, la température, la vibration, la durée d'utilisation, la vitesse du moteur, une éventuelle erreur et un horodatage. Le backend s'abonne au motif `fablab/machines/+ /data`, analyse les messages JSON et enregistre les mesures dans la base de données SQLite.

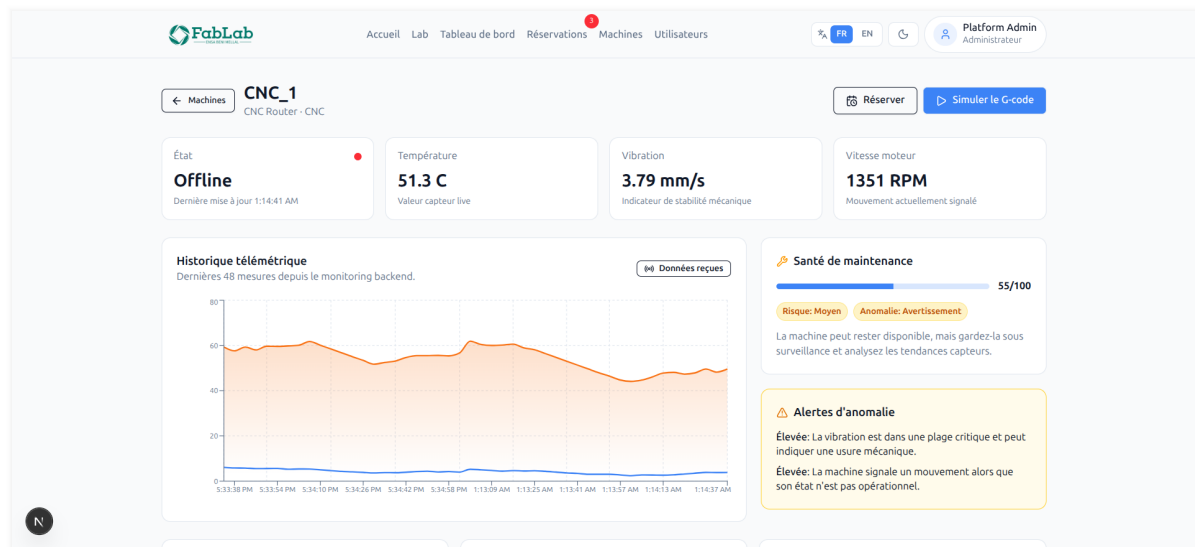


Figure 3.10: Flux MQTT implémenté dans la plateforme

3.14 Tableau de bord de supervision

Le tableau de bord administrateur centralise les informations de supervision. Il affiche l'état de la connexion MQTT, la fraîcheur des données, le nombre d'utilisateurs, le nombre de machines, les réservations en attente et les résultats de monitoring intelligent.

Sur le plan de l'implémentation, le frontend interroge les endpoints administrateur du backend afin d'afficher les résultats calculés par les services d'analyse. Les éléments restitués incluent notamment :

- un statut d'anomalie par machine ;
- un score de santé sur 100 ;
- un niveau de risque de maintenance ;
- une probabilité indicative de risque ;
- une recommandation de maintenance ;
- des facteurs explicatifs tels que température, vibration, durée d'utilisation, vitesse du moteur et erreurs récentes.

Les principes scientifiques, les caractéristiques utilisées, les modèles Machine Learning et les limites de validation sont détaillés dans le chapitre suivant afin d'éviter la duplication entre l'implémentation logicielle et la contribution analytique.

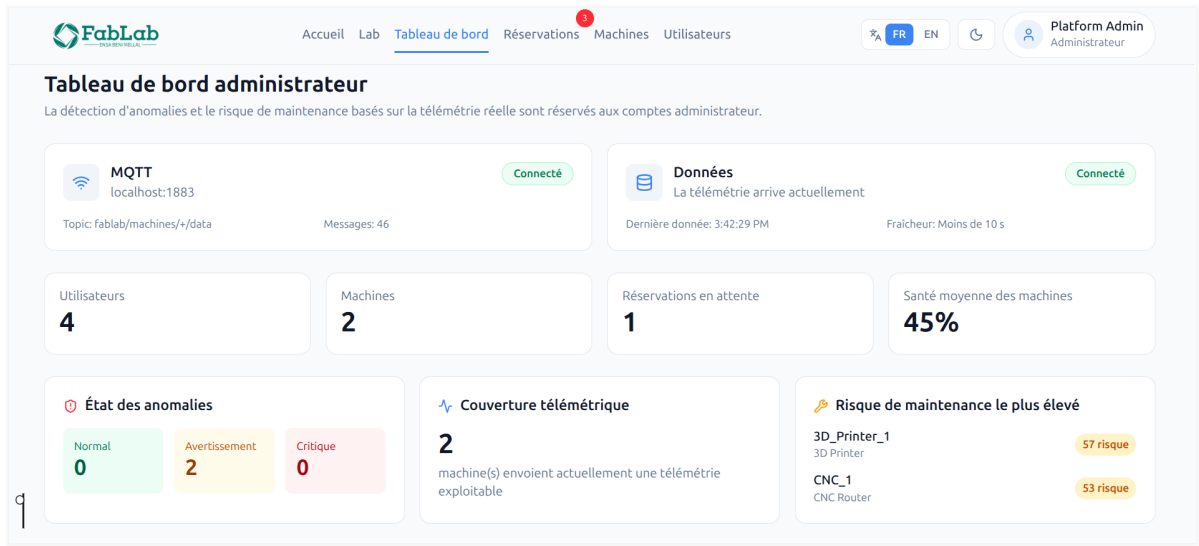


Figure 3.11: Tableau de bord administrateur et supervision intelligente

3.15 Cartes de prédiction et historique de télémétrie

Les cartes de prédiction du tableau de bord permettent de consulter rapidement les machines présentant un besoin de surveillance. Elles affichent les résultats déjà calculés par le backend : score de santé, probabilité indicative, niveau de risque et recommandation synthétique. Cette partie relève de l'intégration applicative, tandis que le détail des modèles est présenté séparément dans le chapitre consacré au module Data Science et IA.



Figure 3.12: Cartes de prédiction et score de santé des machines

Les pages de détail des machines exploitent l'historique des mesures stockées pour afficher l'évolution de certains indicateurs, notamment la température. Cette visualisation permet à l'administrateur de suivre l'état récent d'une machine et de relier les alertes à une série temporelle.



Figure 3.13: Historique des mesures d'une machine

3.16 Intégration technique du Copilote IA

L'espace administrateur intègre une interface flottante de type chatbot appelée Assistant IA de Maintenance. Cette interface est chargée dans le layout de l'application, mais elle n'est affichée que lorsque l'utilisateur connecté possède le rôle `admin`. Elle conserve l'historique de conversation localement dans l'état React du composant et envoie les questions au backend à travers l'endpoint `/admin/ai/copilot/query`.

Le frontend affiche la réponse structurée retournée par le backend : intention reconnue, niveau de sévérité, confiance, texte de synthèse, points de données et recommandations. Cette intégration fournit un accès conversationnel aux informations du tableau de bord, sans transformer le système en modèle génératif. Le fonctionnement scientifique du classifieur d'intentions et la génération déterministe des réponses sont détaillés dans le chapitre suivant.

3.17 Communication entre frontend et backend

La communication entre le frontend et le backend est centralisée dans une fonction utilitaire d'appel API. Cette fonction construit l'URL du backend, ajoute l'en-tête `Content-Type`, insère le jeton d'authentification lorsque nécessaire et traite les erreurs HTTP.

Cette couche simplifie l'usage des API dans les pages Next.js. Elle permet également de maintenir une cohérence dans la gestion des réponses, des erreurs et des routes protégées.

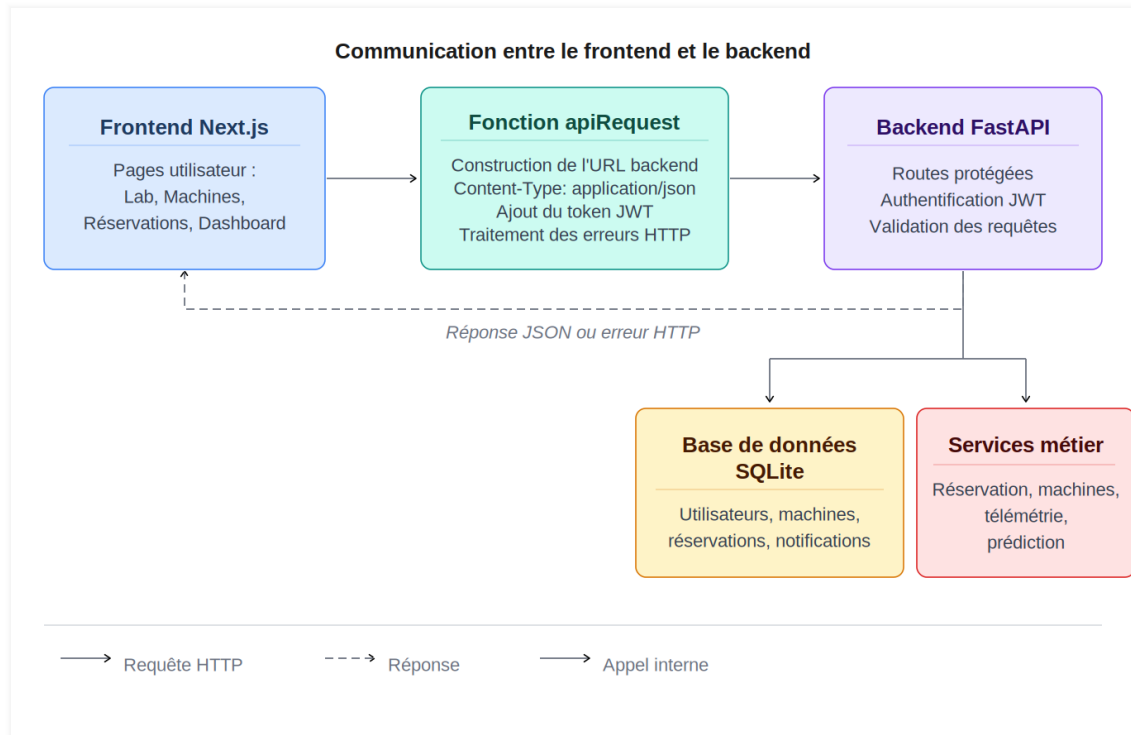


Figure 3.14: *Communication entre le frontend et le backend*

3.18 Interfaces réalisées

Les interfaces réalisées couvrent les principaux parcours utilisateurs : accueil, connexion, inscription, laboratoire 3D, simulation, réservation, profil, gestion des machines, gestion des utilisateurs, réservations administrateur et tableau de bord. Les captures présentées dans les sections précédentes illustrent les écrans les plus représentatifs de ces parcours.

3.19 Conclusion

Ce chapitre a présenté la réalisation technique de la plateforme Virtual FabLab. L'implémentation couvre la gestion des utilisateurs, des machines et des réservations, la visualisation du laboratoire en 3D, la simulation CNC et imprimante 3D, l'ingestion MQTT, le tableau de bord de supervision et l'intégration technique du Copilote IA.

Le chapitre suivant approfondira l'exploitation des données IoT et l'intégration de la Data Science et de l'IA, en détaillant les données utilisées, les modèles de détection, la classification d'intentions, les recommandations de maintenance et les limites de l'approche adoptée.

CHAPITRE 4

MODULE DATA SCIENCE ET INTELLIGENCE ARTIFICIELLE

4.1 Introduction

Ce chapitre présente la contribution Data Science et Intelligence Artificielle intégrée à la plateforme **Virtual FabLab**. Contrairement au chapitre précédent, centré sur la réalisation logicielle, cette partie analyse la chaîne de traitement des données, les modèles utilisés, le classifieur d'intentions et le copilote d'aide à la décision réservé à l'administrateur.

Le module étudié repose sur des données de télémétrie simulées. Il ne constitue donc pas une validation industrielle de maintenance prédictive, mais un prototype académique permettant de démontrer l'intégration entre ingestion MQTT, stockage, apprentissage automatique, classification supervisée d'intentions et restitution des résultats dans l'interface d'administration.

4.2 Architecture du module Data Science et IA

Le module Data Science et IA est placé côté backend. Il reçoit les mesures stockées après ingestion MQTT, extrait les caractéristiques nécessaires, charge les modèles persistés avec `joblib`, puis produit des résultats structurés consommés par le tableau de bord et par le Copilote IA de maintenance.

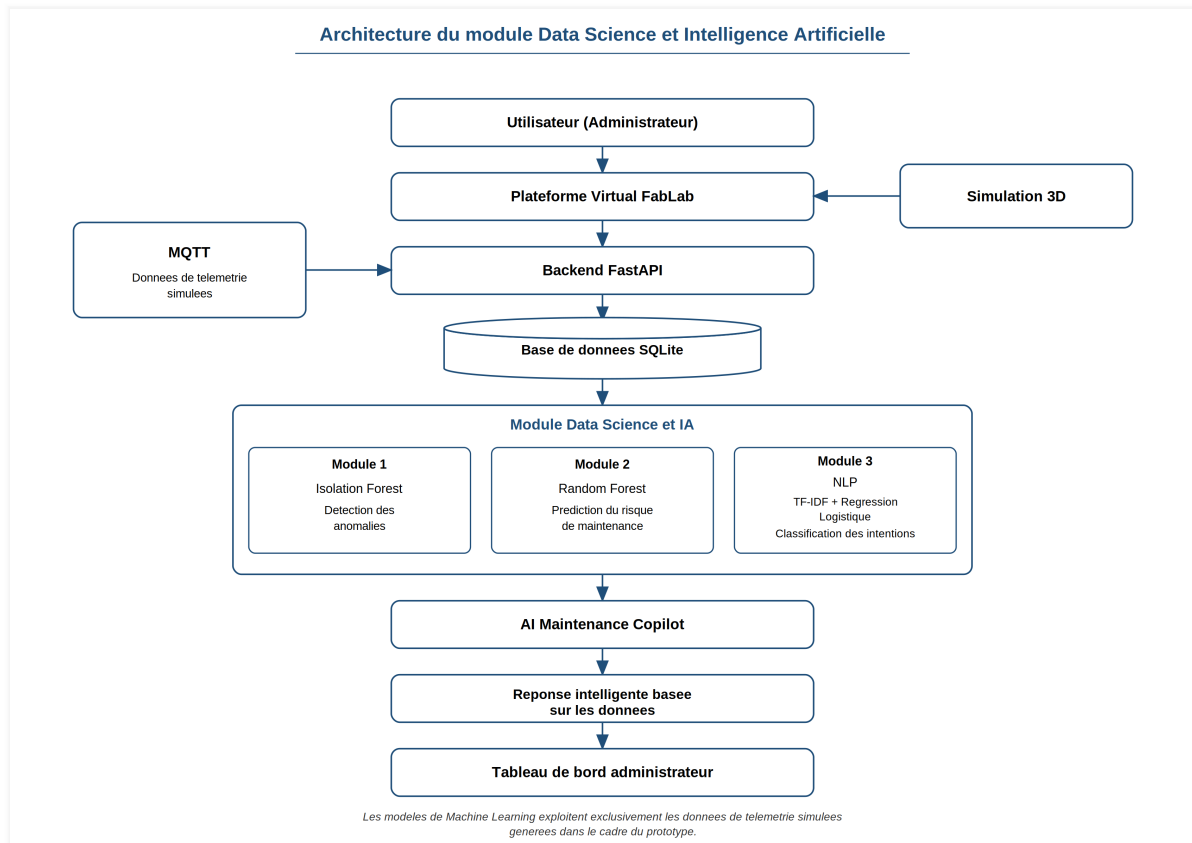


Figure 4.1: Pipeline Data Science et IA de la plateforme

La figure 4.1 doit représenter le passage des messages MQTT vers la base de données, puis vers les traitements de prétraitement, d'ingénierie des caractéristiques, de détection d'anomalies, d'estimation du risque et de restitution dans le tableau de bord et le copilote.

4.3 Collecte et nature des données

Les données exploitées par le module proviennent d'un simulateur MQTT, et non de capteurs physiques connectés. Le script de simulation publie des messages JSON sur des topics propres à chaque machine, selon le motif suivant :

```
fablab/machines/{machine_id}/data
```

Le backend s'abonne au motif `fablab/machines/*/data`, parse les messages reçus et enregistre les mesures dans la table des lectures capteurs.

Les champs effectivement exploités sont la température, la vibration, la durée d'utilisation, la vitesse du moteur, l'erreur éventuelle et l'horodatage. La durée d'utilisation est reçue sous forme de `usage_duration`, puis convertie en heures pour les modèles. La présence d'une erreur est transformée en variable binaire.

Table 4.1: *Nature des données utilisées par le module Data Science*

Donnée	Origine	Utilisation
temperature	MQTT simulé	Détection de dérive thermique, score de risque et facteurs explicatifs.
vibration	MQTT simulé	Détection de vibration anormale et estimation du risque.
motor_speed	MQTT simulé	Contrôle de cohérence et contribution au risque.
usage_duration	MQTT simulé	Conversion en <code>usage_hours</code> pour les modèles et le score de maintenance.
error	MQTT simulé	Historique d'erreurs et variable binaire <code>has_error</code> .
timestamp	MQTT simulé	Ordonnancement temporel et analyse de fraîcheur des mesures.

4.4 Prétraitement des données

Le prétraitement est volontairement limité et cohérent avec l'état actuel du code. Lors de l'ingestion, les messages JSON sont validés par un schéma Pydantic avant d'être persistés.

Pour l'entraînement des modèles Machine Learning, le script `train_ml_models.py` convertit les colonnes numériques en valeurs exploitables, remplace les valeurs manquantes par zéro pour les colonnes attendues et calcule la variable `has_error` à partir du champ `error`.

La conversion temporelle principale consiste à transformer `usage_duration` en `usage_hours`. Le code ne met pas en évidence une normalisation ou une standardisation explicite des variables numériques pour les modèles Isolation Forest et Random Forest. Le rapport ne doit donc pas affirmer l'existence d'une mise à l'échelle qui n'est pas implémentée.

Pour les analyses temps réel, les lectures récentes sont récupérées en ordre décroissant d'horodatage, avec une limite de 48 points pour l'évaluation de maintenance et une fenêtre configurable pour le contexte du copilote.

4.5 Ingénierie des caractéristiques

Les caractéristiques utilisées par les modèles persistés sont définies dans les métadonnées du dossier `backend/ml_models`. Elles sont les suivantes : `temperature`, `vibration`, `usage_hours`, `motor_speed` et `has_error`.

Table 4.2: Entrées utilisées par les modèles Machine Learning

Caractéristique	Description
<code>temperature</code>	Température de fonctionnement issue de la télémétrie simulée.
<code>vibration</code>	Niveau de vibration transmis par le simulateur MQTT.
<code>usage_hours</code>	Durée d'utilisation en heures, dérivée de <code>usage_duration</code> / 60.
<code>motor_speed</code>	Vitesse du moteur, exprimée en tours par minute dans l'interface.
<code>has_error</code>	Variable binaire indiquant la présence d'un code erreur.

En complément des modèles, le service prédictif calcule des facteurs explicatifs par règles : contribution de la température, de la vibration, de la durée d'utilisation, des erreurs récentes et d'une tendance simple basée sur l'évolution moyenne de la température et de la vibration. Ces facteurs servent à rendre les résultats plus interprétables dans le tableau de bord et dans les réponses du copilote.

4.6 Détection des anomalies par Isolation Forest

La détection d'anomalies vise à signaler des comportements inhabituels dans les mesures machines. Le modèle utilisé est un *Isolation Forest*, entraîné avec scikit-learn et persisté dans `backend/ml_models/anomaly_isolation_forest.joblib`.

Le principe d'Isolation Forest consiste à isoler les observations atypiques par des partitions aléatoires. Les points plus faciles à isoler sont considérés comme plus susceptibles d'être anormaux. Dans le script d'entraînement, le modèle est ajusté sur les exemples de classe `normal`. Les paramètres explicitement vérifiés sont :

- `n_estimators=180` ;
- `contamination=0.18` ;
- `random_state=42`.

À l'inférence, le service charge le modèle et appelle `predict` et `decision_function`. La sortie `-1` est interprétée comme une anomalie. Le score retourné est arrondi et exposé dans les réponses de monitoring. Si les fichiers de modèles sont absents ou si une erreur survient, une logique de repli par règles produit une estimation indicative.

Cette détection reste limitée par la nature simulée des données et par l'absence de validation sur machines physiques. Elle doit être interprétée comme une aide à la décision dans le cadre du prototype.

4.7 Évaluation du risque de maintenance par Random Forest

Le risque de maintenance est estimé par un *Random Forest Classifier* scikit-learn, persisté dans `backend/ml_models/maintenance_random_forest.joblib`. Le modèle est entraîné sur les mêmes caractéristiques que l'Isolation Forest, mais avec une cible supervisée appelée `maintenance_label`.

Les classes vérifiées dans les métadonnées sont `normal`, `monitor`, `maintenance_soon` et `critical`. Le script d'entraînement utilise des données synthétiques générées par code et un fichier CSV de démonstration, puis effectue une séparation entraînement/test avec `test_size=0.22`, `random_state=42` et stratification. Les paramètres vérifiés du Random Forest sont `n_estimators=220`, `class_weight="balanced"` et `random_state=42`.

À l'inférence, le service récupère les probabilités de classes avec `predict_proba`. Une probabilité indicative de défaillance est calculée par pondération des classes : `normal` vaut 0,05, `monitor` vaut 0,30, `maintenance_soon` vaut 0,68 et `critical` vaut 0,95. Le score de

santé est ensuite calculé comme `100 - failure_probability * 100`, borné entre 0 et 100.

Cette sortie n'est pas une prédiction industrielle validée. Elle correspond à une estimation du risque sur des données simulées et sert à prioriser la surveillance dans l'interface administrateur.

4.8 Classification NLP des intentions administrateur

Le Copilote IA de maintenance utilise une classification supervisée d'intentions. Il ne s'agit pas d'un modèle génératif ni d'un LLM. Le pipeline d'apprentissage est défini dans le script d'entraînement du classifieur et combine un `TfidfVectorizer` avec une `LogisticRegression`.

Le prétraitement texte vérifié inclut la conversion en minuscules, la suppression des accents avec `strip_accents="unicode"`, l'utilisation de n-grammes de mots de taille 1 et 2, `min_df=1`, `sublinear_tf=True` et un motif de token compatible avec certains identifiants de machines. Le classifieur Logistic Regression utilise `max_iter=1000`, `class_weight="balanced"`, `solver="lbfgs"`, `C=4.0` et `random_state=42`.

Les intentions supportées sont les suivantes :

- `fablab_summary`, `machine_status`, `explain_machine_health` ;
- `highest_risk_machine`, `compare_machines`, `recent_anomalies` ;
- `maintenance_recommendation`, `reservation_summary`, `help` et `unknown`.

Le modèle est persisté dans le fichier `copilot_intent_classifier.joblib`. À l'inférence, une intention est acceptée seulement si la confiance atteint le seuil 0.20; sinon, l'intention devient `unknown`.

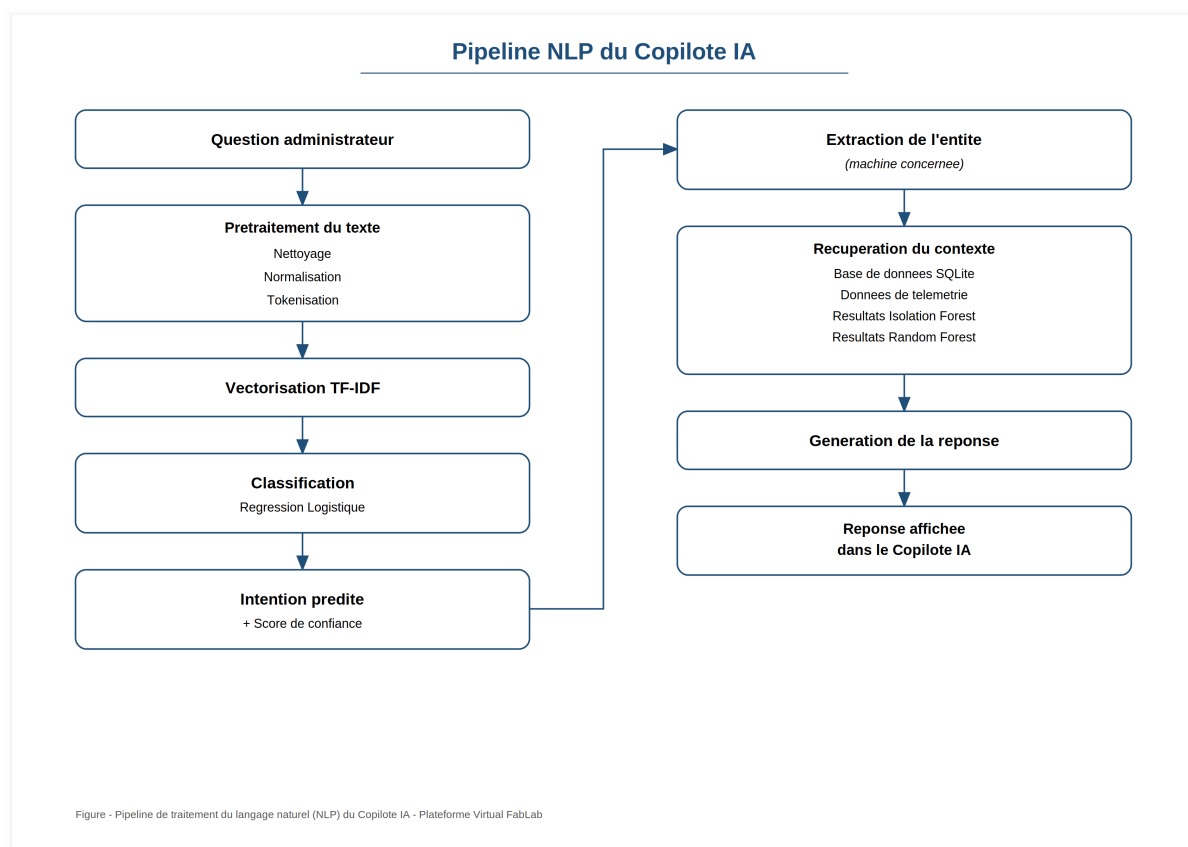


Figure 4.2: *Workflow NLP du Copilote IA de maintenance*

4.9 Copilote IA de maintenance

Le Copilote IA de maintenance est un copilote d'aide à la décision combinant détection d'anomalies, prédiction du risque, classification supervisée des intentions et génération déterministe de réponses fondées sur les données. Il répond à des questions administrateur sur l'état global du FabLab, l'état d'une machine, les anomalies récentes, la machine la plus risquée, les recommandations de maintenance, les réservations et la comparaison de machines.

Le service commence par construire un contexte à partir de la base de données : liste des machines, synthèse de flotte, évaluations de maintenance, historique de télémétrie et résumé des réservations. L'extraction d'entités repère les machines par leur nom, leurs variantes avec tirets ou espaces, ainsi que les références de type `machine 1` ou `#1`. La réponse retournée contient l'intention, la sévérité, la confiance, les machines extraites, des points de données et des recommandations.

Les réponses ne sont pas générées par un modèle de langage. Elles sont construites par des fonctions déterministes et par des textes localisés, à partir des faits observés, des scores calculés et des recommandations par règles. Cette distinction est importante : le copilote

ne produit pas un diagnostic industriel autonome, mais une synthèse structurée d’aide à la décision.

Table 4.3: *Capacités supportées par le Copilote IA de maintenance*

Capacité	Objectif analytique
Synthèse du FabLab	Résumer l’état global de la flotte, le nombre de machines avec télémétrie, les niveaux d’anomalie et les réservations en attente.
État d’une machine	Restituer les faits observés pour une machine donnée : statut, mesures récentes, score de santé et niveau de risque.
Explication du score de santé	Identifier les principaux facteurs contribuant au risque, notamment température, vibration, durée d’utilisation, erreurs et tendance récente.
Anomalies récentes	Lister les machines présentant un statut d’avertissement ou critique, afin de prioriser la surveillance.
Risque le plus élevé	Repérer la machine dont le score de risque de maintenance est le plus important dans la flotte.
Recommandation de maintenance	Fournir une action indicative fondée sur les anomalies, les facteurs de risque et les règles de recommandation.
Comparaison de machines	Comparer deux machines à partir de leurs scores de santé et de risque, sans établir de diagnostic industriel.
Synthèse des réservations	Présenter le nombre de réservations totales, en attente, approuvées, rejetées et prévues le jour même.

4.10 Intégration dans la plateforme

L’intégration backend est exposée par le routeur FastAPI `/admin/ai`. Les endpoints vérifiés sont notamment :

- `/admin/ai/monitoring/overview` ;
- `/admin/ai/monitoring/machines/{machine_id}` ;
- `/admin/ai/copilot/query` et `/admin/ai/copilot` ;
- `/admin/ai/model-status`, `/admin/ai/predict-sample` et `/admin/ai/train`.

Le routeur applique la dépendance `require_admin`, ce qui réserve l’accès aux utilisateurs administrateurs.

Côté frontend, le composant `AiCopilotWidget` vérifie que l'utilisateur connecté possède le rôle `admin`. Il affiche un bouton flottant et un panneau de conversation, puis envoie les questions vers `/admin/ai/copilot/query`. L'historique de conversation est conservé localement dans l'état React du composant et n'est pas persisté dans la base de données.

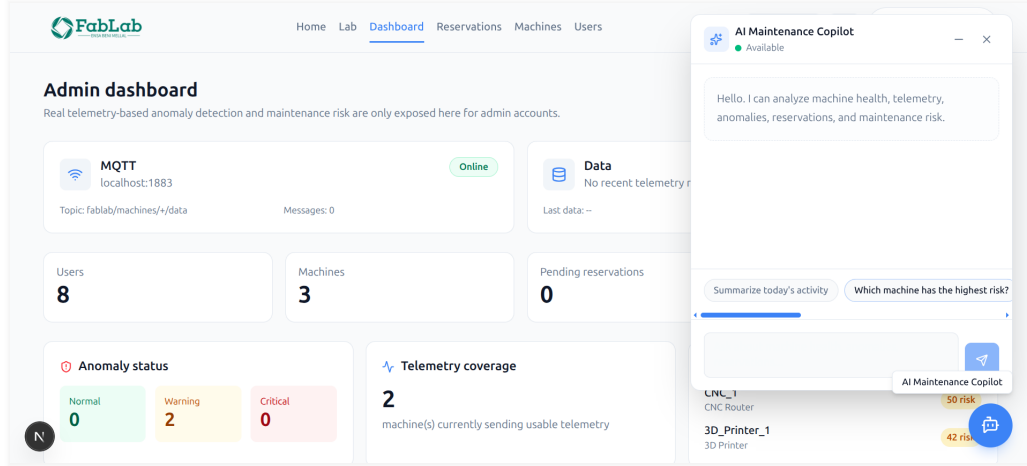


Figure 4.3: Interface flottante du Copilote IA de maintenance

4.11 Évaluation et validation

Les métriques disponibles concernent le classifieur NLP d'intentions. Le fichier de rapport du classifieur indique un jeu de 124 questions étiquetées, séparé en 86 exemples d'entraînement, 19 exemples de validation et 19 exemples de test.

Table 4.4: Métriques disponibles pour la classification NLP des intentions

Métrique	Validation	Test
Accuracy	0,4211	0,4211
Précision macro	0,4867	0,3500
Rappel macro	0,4833	0,4500
F1-score macro	0,4486	0,3800
Taille de l'échantillon	19	19

Ces résultats doivent être interprétés avec prudence, car le jeu de test est réduit. Ils montrent surtout que le pipeline d'évaluation existe et que les métriques sont sauvegardées, mais ils ne suffisent pas pour conclure à une robustesse linguistique élevée.

L'évaluation actuelle du module Data Science et IA porte principalement sur le comportement en inférence, la cohérence des sorties produites, l'intégration dans la plateforme et les métriques disponibles pour le classifieur NLP. Cette validation permet de vérifier le

fonctionnement global du prototype et la bonne exploitation des données simulées. Une validation quantitative plus large, appuyée sur des jeux de test indépendants et sur des données réelles, constitue toutefois une étape complémentaire nécessaire pour apprécier plus finement la robustesse des modèles dans un contexte opérationnel.

Table 4.5: *Synthèse de l'évaluation du module Data Science et IA*

Composant	Validation réalisée	Validation complémentaire proposée
Isolation Forest	Chargement du modèle, production du score d'anomalie, tests d'inférence et intégration dans la plateforme.	Validation sur données réelles, étude de stabilité et comparaison avec des scénarios connus.
Random Forest	Vérification des variables d'entrée, des classes et des prédictions produites.	Évaluation quantitative sur un jeu de test indépendant et plus large.
Classifieur NLP	Accuracy, précision, rappel, F1-score et matrice de confusion.	Enrichissement du corpus, formulations multilingues et validation sur davantage de questions.
Copilote IA	Tests d'accès administrateur, classification d'intentions, extraction d'entités et génération de réponses fondées sur les données.	Évaluation utilisateur et analyse qualitative des réponses.

4.12 Limites

Les limites principales du module sont liées à la nature des données et au périmètre du prototype. La télémétrie est simulée, ce qui permet de tester les flux et les interfaces, mais ne remplace pas une collecte issue de capteurs installés sur des machines réelles. Les modèles entraînés ne sont donc pas validés industriellement.

Le jeu de données NLP reste limité et les métriques actuelles montrent une performance modeste sur un petit échantillon de test. Le système fonctionne comme un classifieur d'intentions supervisé, pas comme un agent conversationnel génératif. Les réponses sont déterministes et fondées sur les données disponibles.

Enfin, les recommandations doivent être comprises comme une aide à la décision. Elles distinguent les faits observés, les scores inférés et les actions conseillées, sans prétendre diagnostiquer avec certitude une panne mécanique ou électrique.

4.13 Conclusion du chapitre

Ce chapitre a isolé la contribution Data Science et IA du projet Virtual FabLab. Le module combine l'ingestion de données MQTT simulées, l'extraction de caractéristiques, la détection d'anomalies par Isolation Forest, l'estimation du risque par Random Forest, la classification supervisée des intentions par TF-IDF et Logistic Regression, ainsi qu'un Copilote IA de maintenance réservé aux administrateurs.

L'ensemble constitue une couche d'aide à la décision adaptée à un prototype académique. Les résultats doivent cependant rester contextualisés : les données sont simulées, les métriques Machine Learning industrielles sont absentes pour les modèles de maintenance, et une validation future sur données réelles reste nécessaire.

CHAPITRE 5

VALIDATION ET ÉVALUATION DU SYSTÈME

5.1 Introduction

Ce chapitre présente la validation de la plateforme **Virtual FabLab** selon plusieurs axes complémentaires. L’objectif est d’évaluer le comportement du système réalisé, depuis les parcours fonctionnels jusqu’à l’intégration du module Data Science et Intelligence Artificielle, en cohérence avec l’architecture décrite dans les chapitres précédents.

L’évaluation porte ainsi sur la validation fonctionnelle de l’interface et du backend, la communication entre le frontend et l’API, la réception de la télémétrie MQTT, la persistance des données, la simulation 3D, les tests d’inférence des modèles Machine Learning, la classification d’intentions et l’intégration du Copilote IA de maintenance. Les résultats sont interprétés dans le cadre d’un environnement de test académique, sans prétendre à une validation industrielle sur machines physiques.

5.2 Stratégie de validation

La stratégie adoptée consiste à vérifier le système à plusieurs niveaux. Les fonctionnalités principales sont validées à travers des scénarios d’utilisation, des vérifications d’interface, des tests backend, des contrôles de flux MQTT, des consultations en base de données, des comportements de simulation et des tests d’intégration du module Data Science et IA.

Trois types de validation sont distingués. La validation fonctionnelle vérifie que les parcours utilisateur et administrateur produisent les comportements attendus. L’évaluation chiffrée concerne les composants pour lesquels des métriques sont disponibles, en particulier le classifieur NLP. Enfin, la validation qualitative d’intégration vérifie que les modules communiquent correctement : télémétrie, analyse, tableau de bord et Copilote IA.

Cette démarche permet d’apprécier la cohérence du prototype tout en conservant une distinction claire entre tests fonctionnels, tests d’intégration, validation d’inférence des modèles et évaluation quantitative.

5.3 Validation fonctionnelle de la plateforme

5.3.1 Validation du frontend

La validation du frontend a concerné les interfaces développées avec Next.js. Les scénarios vérifiés portent sur la page de connexion, l’inscription, la navigation selon le rôle, la consultation des machines, la gestion des réservations, l’affichage des notifications, le tableau de bord de télémétrie et l’accès à l’interface de simulation 3D.

Les pages réservées à l'administration sont accessibles uniquement après authentification et selon le rôle de l'utilisateur. Dans l'environnement de test, les redirections vers la page de connexion et l'affichage conditionnel des modules administrateur correspondent au comportement attendu. Les pages liées aux machines et aux réservations permettent également de vérifier la cohérence des informations retournées par le backend.

La figure 5.1 illustre l'utilisation du tableau de bord pour centraliser les mesures, les indicateurs de supervision et les résultats liés au module Data Science.

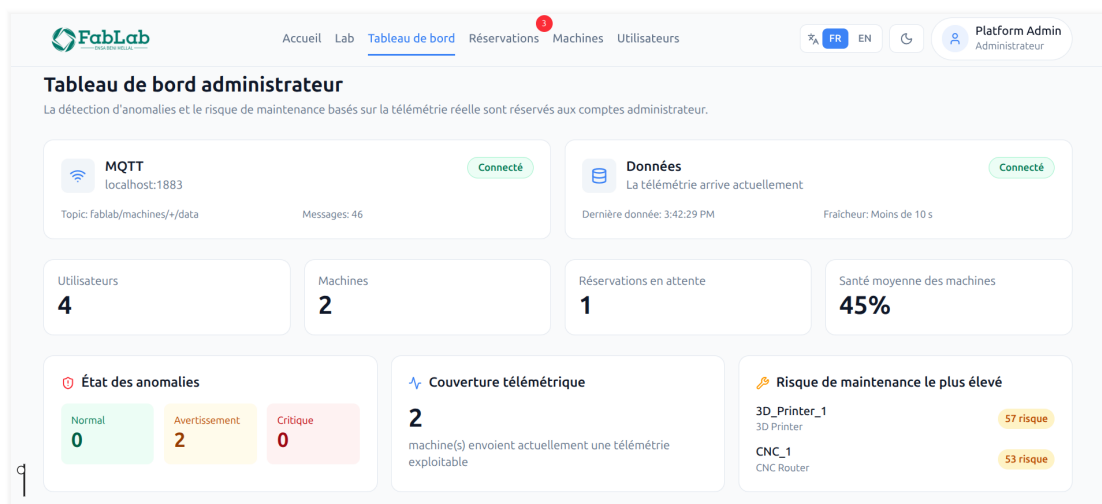


Figure 5.1: Tableau de bord de supervision et de télémétrie

5.3.2 Validation du backend

La validation du backend FastAPI a porté sur l'authentification, la protection JWT, les droits d'accès, les opérations de gestion et les réponses des endpoints. Les tests existants vérifient notamment le refus d'un jeton invalide, l'interdiction d'accès administrateur pour un utilisateur étudiant, l'accès autorisé pour un administrateur et la disponibilité des routes du Copilote IA.

Les opérations de gestion des réservations ont également été vérifiées. Les tests backend couvrent le comptage des réservations en attente, le passage d'une réservation à l'état approuvé et la mise à jour du compteur associé. Ces vérifications confirment que la logique de traitement des réservations reste cohérente avec les données persistées.

La communication entre l'interface et le backend repose sur des requêtes HTTP structurées, centralisées côté frontend par une fonction utilitaire d'appel API. La figure 5.2 synthétise ce principe : l'interface transmet la requête, le backend applique les contrôles nécessaires, interagit avec la base de données ou les services concernés, puis retourne une réponse JSON ou une erreur exploitable par l'interface.

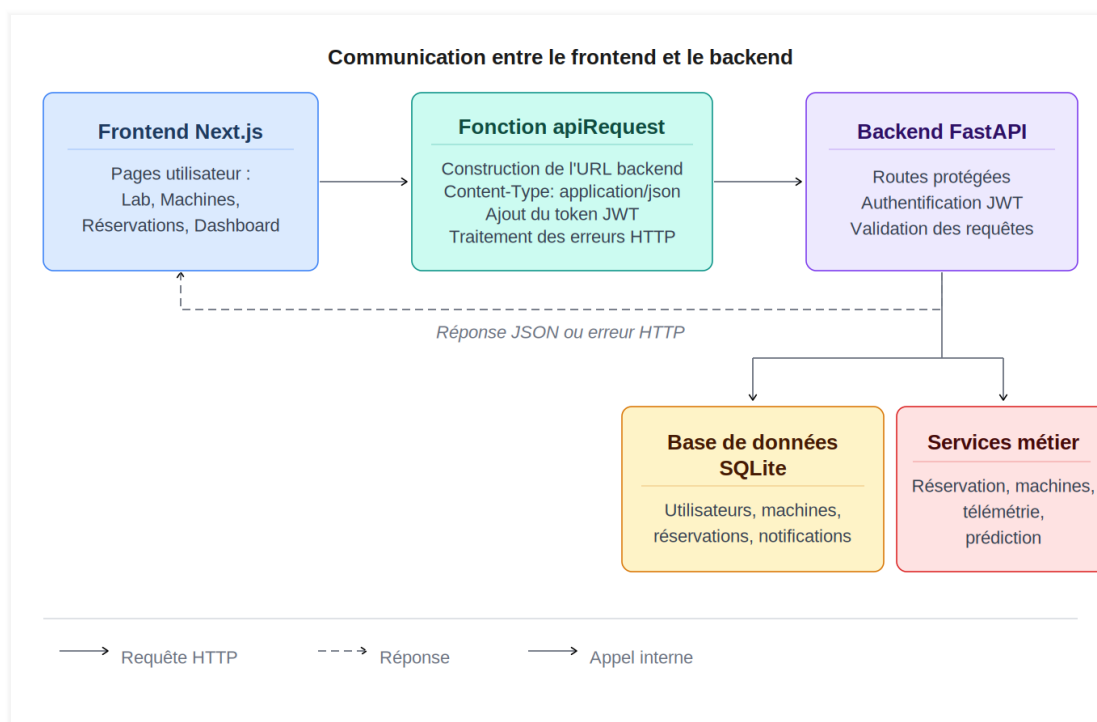


Figure 5.2: *Communication entre l'interface Next.js et le backend FastAPI*

5.3.3 Validation de la communication MQTT et de la télémétrie

La validation de la télémétrie concerne le flux suivant : simulateur MQTT, broker, backend, base de données, puis tableau de bord. Dans l'environnement de test, la télémétrie est simulée afin de vérifier le fonctionnement de la chaîne applicative avant une éventuelle connexion à des capteurs physiques.

Les messages publiés contiennent les champs effectivement utilisés par l'application : `machine_id`, `temperature`, `vibration`, `usage_duration`, `motor_speed`, `error`, `timestamp` et `status`. Le backend s'abonne au topic MQTT configuré, parse les messages JSON, identifie la machine concernée et persiste les mesures dans la table des lectures capteurs.

Les tests réalisés vérifient la réception des messages, l'association des mesures à une machine, la persistance des lectures, la consultation du dernier état machine et l'affichage des informations dans le tableau de bord. Les informations d'erreur ou d'anomalie peuvent ensuite être exploitées par les services de monitoring et par le Copilote IA.

5.3.4 Validation de la simulation 3D

La simulation 3D a été validée sur les deux équipements principaux du FabLab virtuel : l'imprimante 3D et la machine CNC. Les vérifications portent sur le chargement de la machine, l'interaction caméra, l'import de fichiers compatibles, l'interprétation des instructions, le mouvement progressif de l'outil et la visualisation de la trajectoire.

Pour l'imprimante 3D, la validation concerne l'analyse de fichiers G-code, la progression de la tête d'impression, l'affichage de la position selon les axes, la visualisation progressive du dépôt et les contrôles de lecture, pause et réinitialisation. Pour la CNC, les tests portent sur l'interprétation de fichiers NC ou G-code compatibles, l'affichage du parcours d'outil, l'activation du mouvement de coupe, l'estimation du trajet et les contrôles d'animation.

La figure 5.3 présente l'interface de simulation utilisée pour observer les comportements validés. Cette simulation a une finalité pédagogique et interactive ; elle ne constitue pas une validation physique complète du comportement réel des machines.

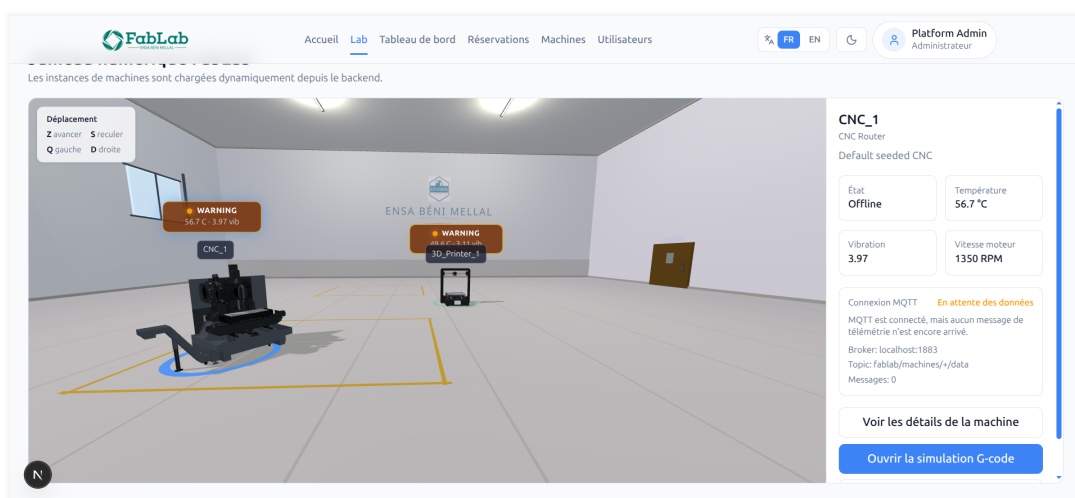


Figure 5.3: Interface de simulation 3D du FabLab virtuel

5.4 Validation du module Data Science et IA

5.4.1 Validation de l'Isolation Forest

La validation de l'Isolation Forest vise à vérifier le chargement du modèle, la préparation des entrées et la production d'un statut d'anomalie exploitable par le monitoring. Les variables utilisées sont celles définies dans le chapitre précédent : température, vibration, durée d'utilisation convertie en heures, vitesse du moteur et présence éventuelle d'une erreur.

À l'inférence, le service applique le modèle chargé depuis l'artefact `joblib`, produit un indicateur d'anomalie et retourne un score associé. Ces sorties sont intégrées à l'évaluation de l'état machine et au tableau de bord. La validation porte donc sur le comportement d'inférence et l'intégration dans le workflow de supervision, sans introduire de métriques de classification non disponibles pour ce composant.

5.4.2 Validation du Random Forest

Le Random Forest est utilisé pour estimer le risque de maintenance à partir des mêmes variables d'entrée. La validation a porté sur la cohérence des entrées, la disponibilité des classes déclarées dans les métadonnées, la production d'une prédiction et l'exploitation de cette prédiction par l'application.

Les sorties utilisées par la plateforme comprennent notamment le statut de maintenance, une probabilité indicative de défaillance, un score de santé et une recommandation. Dans le cadre actuel, ces résultats servent à prioriser la surveillance et à enrichir le tableau de bord. Aucune métrique indépendante supplémentaire n'est ajoutée ici ; une évaluation quantitative sur un jeu de test plus large et indépendant reste une perspective de validation complémentaire.

5.4.3 Validation de la classification NLP

La classification NLP du Copilote IA repose sur un pipeline classique combinant TF-IDF et Logistic Regression. L'objectif de la validation est de vérifier la capacité du classifieur à associer une question administrateur à une intention supportée, sans le présenter comme un modèle génératif ou un grand modèle de langage.

Le fichier d'évaluation disponible indique un corpus de 124 questions étiquetées, séparées en 86 exemples d'entraînement, 19 exemples de validation et 19 exemples de test. Les métriques disponibles sont présentées dans le tableau 5.1.

Table 5.1: *Résultats d'évaluation du classifieur d'intentions*

Métrique	Validation	Test
Accuracy	0,4211	0,4211
Précision macro	0,4867	0,3500
Rappel macro	0,4833	0,4500
F1-score macro	0,4486	0,3800

Ces résultats valident l'existence d'une chaîne NLP complète : constitution du corpus, vectorisation, apprentissage, séparation des données, calcul des métriques et matrice de confusion. Les performances restent liées à la taille et à la diversité du corpus ; leur amélioration pourra passer par l'ajout de formulations, de questions multilingues et de scénarios administrateur plus variés.

5.4.4 Validation du Copilote IA de maintenance

La validation du Copilote IA concerne son intégration applicative et son comportement dans les scénarios administrateur. Les tests existants vérifient l'accès réservé à l'administrateur, le refus d'un utilisateur non autorisé, la soumission d'une question, la classification d'intention, l'extraction d'entités machines, la récupération du contexte et la génération d'une réponse fondée sur les données disponibles.

Les scénarios testés couvrent notamment la synthèse globale du FabLab, l'explication de l'état d'une machine, la recherche de la machine la plus risquée, la comparaison de machines, les anomalies récentes, les recommandations de maintenance et les questions non supportées. Le Copilote repose sur la classification d'intentions, le contexte métier, les données de télémétrie et les sorties des modèles ; il ne produit pas de raisonnement génératif libre.

La figure 5.4 montre l'interface représentative retenue pour illustrer l'intégration du Copilote IA dans le tableau de bord administrateur.

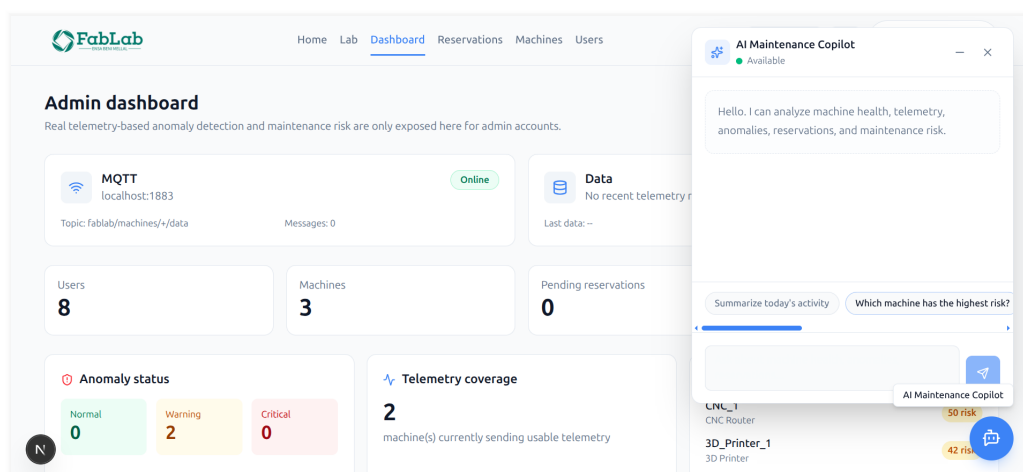


Figure 5.4: Interface du Copilote IA de maintenance réservé à l'administrateur

Table 5.2: *Synthèse de la validation du système*

Composant	Scénario de validation	Résultat observé	Statut
Authentification	Connexion, génération du jeton JWT et accès au profil.	Les routes protégées nécessitent un jeton valide.	Validé
Gestion des utilisateurs	Accès différencié selon le rôle utilisateur.	Les accès administrateur sont restreints.	Validé
Gestion des machines	Consultation et gestion des machines depuis l'interface et l'API.	Les données machines sont exposées et exploitables.	Fonctionnel dans l'environnement de test
Réservations	Création, approbation et comptage des réservations en attente.	Les états et compteurs sont mis à jour correctement.	Validé
Notifications	Génération et affichage des informations liées aux actions utilisateur.	Les notifications accompagnent les parcours principaux.	Intégré et testé
Communication MQTT	Publication, réception et parsing des messages de télémétrie.	Les messages valides sont reçus et traités.	Fonctionnel dans l'environnement de test
Stockage de la télémétrie	Persistence des mesures associées aux machines.	Les lectures sont consultables par les endpoints de monitoring.	Validé
Tableau de bord	Affichage des états machines, scores et recommandations.	Les indicateurs de supervision sont centralisés.	Intégré et testé
Simulation imprimante 3D	Chargement du modèle, lecture G-code et animation progressive.	La trajectoire et la progression sont visibles.	Fonctionnel dans l'environnement de test
Simulation CNC	Lecture de fichiers compatibles, parcours d'outil et contrôles d'animation.	La simulation du mouvement d'outil est observable.	Fonctionnel dans l'environnement de test
Isolation Forest	Chargement du modèle et production d'un statut d'anomalie.	Le résultat est intégré au monitoring.	Intégré et testé
Random Forest	Vérification des entrées, classes et prédictions de risque.	Les sorties alimentent le score et les recommandations.	Intégré et testé
Classifieur NLP	Calcul des métriques et test des intentions supportées.	Les intentions sont évaluées sur validation et test.	Validé
Copilote IA	Accès administrateur, question, intention, entités et réponse contextualisée.	Les réponses restent fondées sur le contexte disponible.	Intégré et testé

5.5 Discussion des résultats

Les résultats obtenus montrent que les principaux objectifs du projet ont été atteints dans le cadre de l'environnement étudié. La plateforme combine une interface web, un backend FastAPI, une base de données, un flux de télémétrie MQTT, une simulation 3D et un module Machine Learning au sein d'une architecture cohérente.

L'intégration web permet de centraliser les fonctions de gestion du FabLab virtuel intel-

ligent : machines, réservations, utilisateurs, notifications et supervision. Cette centralisation facilite le suivi des équipements et offre à l’administrateur une vue structurée de l’état du système.

La partie Data Science et IA apporte une couche d’aide à la décision. La détection d’anomalies et la prédiction du risque de maintenance permettent d’orienter l’attention vers les machines nécessitant une surveillance particulière. Le Copilote IA de maintenance complète cette approche en offrant une interaction textuelle guidée, fondée sur des intentions, des entités machines et des données extraites de la plateforme.

Les limites doivent toutefois être interprétées avec mesure. La télémétrie utilisée dans l’évaluation est simulée, le système n’a pas encore été déployé sur des machines réelles de FabLab, le corpus NLP reste limité et les modèles de maintenance nécessitent des jeux de données indépendants plus larges pour une évaluation quantitative complète. De plus, le comportement simulé peut différer du comportement physique de machines réelles, notamment pour les contraintes mécaniques, thermiques ou vibratoires.

Ces éléments ne remettent pas en cause la cohérence du prototype. Ils indiquent plutôt les conditions nécessaires pour étendre l’évaluation. Les résultats constituent une validation de faisabilité dans l’environnement étudié et démontrent l’intégration technique des composants proposés, tandis qu’une validation plus large sur données réelles demeure une perspective naturelle.

5.6 Résultats globaux

Le projet aboutit à une plateforme web fonctionnelle permettant la consultation et l’administration d’un FabLab virtuel intelligent. Les rôles utilisateur et administrateur sont distingués, les machines et les réservations peuvent être gérées, et les informations de télémétrie sont ingérées, stockées et visualisées.

La simulation 3D permet de représenter les équipements supportés, notamment l’impression 3D et la CNC, avec des contrôles d’animation et une visualisation progressive des trajectoires. Cette dimension renforce l’objectif pédagogique du projet en rendant les processus de fabrication plus accessibles.

Le module Data Science et IA intègre la détection d’anomalies par Isolation Forest, la prédiction du risque de maintenance par Random Forest, la classification d’intentions par TF-IDF et Logistic Regression, ainsi qu’un Copilote IA de maintenance réservé à l’administrateur. Ces résultats contribuent à l’objectif général du projet : proposer une plateforme

de FabLab virtuel capable de combiner supervision, simulation et aide à la décision.

5.7 Conclusion

Ce chapitre a présenté la validation du système selon plusieurs niveaux : frontend, backend, communication API, télémétrie MQTT, simulation 3D, modèles Machine Learning, classification NLP et Copilote IA de maintenance. Les vérifications réalisées confirment la cohérence de l'architecture intégrée et l'atteinte des principaux objectifs fonctionnels du projet.

Les résultats doivent rester contextualisés : la validation repose sur un environnement de test, des données de télémétrie simulées et un corpus NLP encore limité. Une généralisation vers un contexte opérationnel nécessitera une collecte sur machines réelles, des jeux de données plus larges et une évaluation quantitative complémentaire. Ces éléments préparent naturellement la conclusion générale, qui synthétise les apports du projet et ses perspectives d'évolution.

CONCLUSION GÉNÉRALE

Ce projet de fin d'études avait pour objectif de concevoir et de réaliser une plateforme web de FabLab virtuel intelligent, destinée à faciliter la gestion des machines, la visualisation de l'environnement de fabrication et la supervision des équipements dans un contexte académique.

Les travaux réalisés ont permis de mettre en place une plateforme web fonctionnelle intégrant une simulation 3D du FabLab, un système de communication basé sur le protocole MQTT pour la réception des données de télémétrie, une détection d'anomalies, une évaluation du risque de maintenance, une classification supervisée d'intentions et un Copilote IA de maintenance réservé à l'administrateur. Un tableau de bord administrateur complète la solution en assurant le suivi des équipements, des réservations et des principaux indicateurs de supervision.

Dans la version actuelle, les données de télémétrie utilisées sont simulées. Cette approche a permis de valider l'architecture globale de la solution, les flux de communication MQTT, le stockage des données et le fonctionnement du module d'analyse. Les résultats obtenus ne constituent donc pas une validation industrielle, mais une base expérimentale pour une future intégration dans un environnement réel.

Les perspectives de ce projet concernent principalement la connexion de la plateforme à des capteurs et à des machines réelles, la constitution de jeux de données plus larges, le réentraînement des modèles sur de la télémétrie réelle, l'enrichissement du corpus NLP, le support multilingue, l'amélioration de l'explicabilité, l'intégration optionnelle d'un LLM strictement ancré dans les données et la validation dans un FabLab opérationnel.

BIBLIOGRAPHIE ET WEBOGRAPHIE

- [1] Neil Gershenfeld, *Fab: The Coming Revolution on Your Desktop*, Basic Books, 2005.
- [2] Klaus Schwab, *The Fourth Industrial Revolution*, World Economic Forum, 2016.
- [3] Samuel Greengard, *The Internet of Things*, MIT Press, 2015.
- [4] Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly Media, 2019.
- [5] Vercel, *Documentation officielle de Next.js*, <https://nextjs.org/docs>, consultée le 22 mai 2026.
- [6] FastAPI, *Documentation officielle de FastAPI*, <https://fastapi.tiangolo.com/>, consultée le 22 mai 2026.
- [7] Three.js, *Documentation officielle de Three.js*, <https://threejs.org/docs/>, consultée le 22 mai 2026.
- [8] MQTT.org, *Documentation et ressources officielles du protocole MQTT*, <https://mqtt.org/>, consultée le 22 mai 2026.
- [9] SQLite, *Documentation officielle de SQLite*, <https://www.sqlite.org/docs.html>, consultée le 22 mai 2026.
- [10] Auth0, *Introduction aux JSON Web Tokens*, <https://jwt.io/introduction>, consultée le 22 mai 2026.
- [11] Scikit-learn, *Documentation officielle de Scikit-learn*, <https://scikit-learn.org/stable/documentation.html>, consultée le 22 mai 2026.
- [12] Tailwind Labs, *Documentation officielle de Tailwind CSS*, <https://tailwindcss.com/docs>, consultée le 22 mai 2026.
- [13] Blender Foundation, *Documentation officielle de Blender*, <https://docs.blender.org/>, consultée le 22 mai 2026.